ChatGPT

Guida ai Principali Pattern Agentici

Introduzione: Nel campo dell'**Agentic AI**, i *pattern agentici* sono schemi ricorrenti che descrivono **come agenti intelligenti e sotto-agenti possono interagire** per raggiungere obiettivi complessi. Questi pattern forniscono un'architettura strutturata, analoga ai design pattern nel software tradizionale, per orchestrare il ragionamento, la collaborazione e l'azione autonoma degli agenti [1] [2]. Di seguito descriviamo **undici principali pattern agentici**, spiegando il concetto, vantaggi e limiti, e fornendo linee guida di implementazione sia con **Claude (Anthropic)** sia con **OpenAI (GPT-4/Codex)**, includendo esempi di prompt e architetture per ciascun caso.

1. Pattern Pianificatore–Esecutore (Planner–Executor)

Descrizione

Questo pattern **separa il "cosa fare" dal "come farlo"** tramite due ruoli distinti [3]. L'**agente Pianificatore** riceve l'obiettivo generale e lo **decompone in sotto-compiti** o passi concreti. Successivamente, uno o più **agenti Esecutori** eseguono ciascun sotto-compito e restituiscono i risultati al Pianificatore [4]. In pratica, il Pianificatore funge da "regista" che orchestra il lavoro, mentre gli Esecutori sono "operatori" specializzati che svolgono i compiti specifici. Il Pianificatore poi **raccoglie e integra i risultati** degli Esecutori per produrre la soluzione finale [5].

Vantaggi e limiti

Vantaggi: Il pattern Pianificatore–Esecutore è **modulare e scalabile**, facilitando l'aggiunta di nuovi compiti o agenti senza stravolgere l'intero sistema [6]. Permette anche l'**esecuzione concorrente** dei sotto-compiti (ad es. più esecutori in parallelo) e supporta workflow complessi suddividendoli in unità gestibili [6]. Inoltre, ogni agente può essere specializzato nel proprio ambito, migliorando l'efficienza complessiva.
Limiti: Un piano errato o incompleto da parte del Pianificatore **propaga l'errore a cascata** nei sotto-compiti [7]. È quindi cruciale che il Pianificatore elabori correttamente la strategia iniziale. Servono inoltre meccanismi per gestire le **dipendenze** tra compiti e per reagire qualora un esecutore fallisca o restituisca risultati imprevisti [7]. La comunicazione aggiuntiva tra Pianificatore ed Esecutori aumenta la complessità e richiede gestione del contesto condiviso.

Implementazione con Claude (Anthropic)

Con Claude, possiamo implementare questo pattern istanziando **due ruoli conversazionali** separati. Ad esempio, si può iniziare una conversazione con Claude impostando un prompt che definisce il ruolo di *Pianificatore* (nell'istruzione di sistema o nella prima richiesta). Il Pianificatore (Claude) riceve il goal dell'utente e restituisce un elenco numerato di passi o sotto-obiettivi. Per ogni passo, possiamo avviare (sempre con Claude o con un altro istanza) un *sotto-agente Esecutore* passando come prompt il singolo compito da svolgere. In pratica si alterna tra: **Claude-Pianificatore** che genera piani, e **Claude-Esecutore** che, dato un compito, restituisce il risultato. L'aggregazione finale può essere fatta dal Pianificatore concatenando o riassumendo i risultati dei vari esecutori. Claude supporta contesti molto estesi, quindi un approccio pratico è mantenere lo **storico dei task e risultati nella stessa conversazione** (finché rientra nei limiti di token), permettendo al Pianificatore di ricordare le decisioni precedenti. Si possono anche utilizzare le **API di Claude** (ad esempio tramite Anthropic API)

orchestrando questo flusso in codice: prima un chiamata API per ottenere il piano, poi iterare sui passi chiamando Claude per ogni step. Per migliorare la robustezza, il Pianificatore può essere istruito a rivedere il piano in base ai risultati ottenuti (implementando un *loop di feedback*, vedi Pattern AutoGPT più avanti).

Implementazione con OpenAI (GPT-4/Codex)

Usando GPT-4 o modelli Codex, il pattern Pianificatore–Esecutore può essere realizzato sfruttando meccanismi come le **Function Calling API** o orchestrando manualmente le chiamate. Un approccio è utilizzare un **framework come LangChain**: ad esempio, LangChain fornisce un agent chiamato *Plan-and-Execute* che segue esplicitamente questo schema (pianifica con un LLM, poi esegue con uno o più LLM/tool) [4] [5]. In alternativa, si può costruire una semplice pipeline: prima fare una chiamata a GPT-4 con un prompt del tipo *"Sei un pianificatore intelligente. Suddividi questo obiettivo nei passi necessari."* e ottenere la lista dei compiti. Dopodiché, iterare sui compiti: per ciascuno, fare una nuova chiamata GPT (magari lo stesso modello GPT-4 o un modello specializzato come Codex se il compito è di scrittura di codice) fornendo il compito come prompt (eventualmente insieme al contesto necessario) e ottenere la soluzione parziale. Questi risultati possono essere memorizzati e poi passati di nuovo a GPT-4 per un *passo finale* in cui, con un prompt tipo *"Integra i seguenti risultati dei sottocompiti per ottenere la soluzione finale"*, l'agente aggrega il tutto. GPT-4 eccelle nella pianificazione logica, ma per compiti di esecuzione specifici (es. calcolo, coding) modelli come Codex o funzioni tool potrebbero essere preferibili: ad esempio, se un passo richiede calcoli, si può definire una *function* "calcola(x)" e lasciare che GPT-4 la invochi. **Esempio di prompt (pianificazione)**: *"Obiettivo: Ordinare un viaggio in 3 città europee restando sotto un budget prefissato. Come pianificatore, elenca tutti i passi necessari (p.es. ricerca voli, calcolo costi, prenotazioni) in modo dettagliato."* → GPT-4 restituirà un elenco numerato di sotto-task. **Esempio di prompt (esecuzione)** per un sotto-task: *"Compito: trovare i voli più economici da Roma a Parigi il 10 agosto. Fornisci la soluzione come elenco di opzioni con prezzi."*.

Esempi

Un tipico scenario d'uso è la **pipeline di data processing**: il Pianificatore (LLM) suddivide la pipeline (es. *1. Estrarre dati grezzi, 2. Pulire i dati, 3. Analizzare statistiche, 4. Generare report*), dopodiché Esecutori (script o agent LLM specializzati) svolgono ciascuna fase e ritornano il risultato al Pianificatore che combina tutto in un report finale [8]. Un altro esempio è l'**assistente di scrittura**: dato il compito "scrivi un articolo su X", il Pianificatore individua sezioni (introduzione, sezione 1, sezione 2, conclusione), quindi diversi agenti generativi (anche istanze dello stesso modello in parallelo) scrivono le varie sezioni. Infine, il Pianificatore (o un altro agente finale) assembla e uniforma lo stile dell'articolo completo. In letteratura, molte implementazioni seguono questo pattern; ad esempio, **HuggingGPT** usa ChatGPT come pianificatore centrale che scompone il problema e assegna sotto-task a modelli esperti specializzati (visione, audio, ecc.) integrando poi i risultati [9] [10].

2. Pattern Manager–Worker (Gerarchia Multi-Agente)

Descrizione

Il pattern Manager–Worker è ispirato alle **strutture organizzative umane**: introduce un agente **Manager** che **sovrintende più agenti Worker** (lavoratori) specializzati [11] [12]. In sostanza, il Manager riceve l'obiettivo principale e suddivide il lavoro, **assegnando compiti specifici a ciascun Worker** in base alle loro competenze. Durante l'esecuzione, il Manager **monitora lo stato di avanzamento**, raccoglie i risultati dai Worker e li valida, intervenendo se necessario per riallocare compiti o correggere errori [13] [14]. Questo pattern estende Pianificatore–Esecutore su scala più ampia: immaginate un *project manager AI* che gestisce un team di agenti, ognuno esperto in un sotto-dominio.

Vantaggi e limiti

Vantaggi: Permette **specializzazione e parallelismo su larga scala**. Ogni Worker può essere tarato per uno specifico tipo di compito (es. un agente per analisi dati, uno per scrittura creativa, uno per calcolo matematico). Il Manager può **orchestrare compiti in parallelo**, migliorando l'efficienza su incarichi complessi e voluminosi [15]. Questa struttura gerarchica fornisce inoltre un **controllo centrale**: il Manager funge da punto di coordinamento che garantisce che i risultati finali siano coerenti e integrati. **Mima strutture reali** di gestione, risultando intuitiva da comprendere e da debuggare (il Manager logga decisioni, i Worker i risultati) [15].
Limiti: Introduce **overhead di sincronizzazione e comunicazione**. Il Manager deve continuamente tenere traccia dei progressi e questo scambio di messaggi può rallentare il sistema, specialmente se i Worker sono molti [16]. Inoltre, il Manager diventa un **single point of failure**: se commette un errore nell'assegnazione o valutazione, l'intero processo può risentirne. È necessario implementare strategie di validazione robusta, altrimenti il Manager potrebbe accettare output errati dai Worker [16]. Infine, c'è un **costo computazionale**: mantenere più agenti attivi (soprattutto LLM di grandi dimensioni) in parallelo può essere oneroso in termini di risorse.

Implementazione con Claude

Per realizzare un Manager-Worker con Claude, si può sfruttare la **capacità di Claude di seguire istruzioni di ruolo** su più turni. Un approccio è mantenere **un'unica conversazione** in cui Claude impersona il Manager, e simulare i Worker in maniera descrittiva. Ad esempio, si può fornire a Claude un prompt strutturato: prima un elenco di compiti e agenti assegnatari (che funge da piano), poi chiedere a Claude di svolgere il ruolo di Manager e *per ogni compito* generare un output come se venisse dal relativo Worker. In pratica, Claude può "fingere" di essere diversi specialisti: es. *"Agente Finanza: [risultato]", "Agente Marketing: [risultato]"*, e così via, sotto la direzione del Manager. Questa tecnica sfrutta le abilità di *role-playing* di Claude. In alternativa, si possono avviare **più istanze** di Claude: una principale (Manager) e varie istanze subordinate (Worker). Il Manager Claude, dato un obiettivo, emette comandi tipo: *"Worker1, fai X; Worker2, fai Y"*. Il nostro sistema (codice esterno) intercetta questi comandi e li inoltra come prompt agli altri Claude-Worker. I risultati poi vengono riportati al Manager Claude in un messaggio successivo. Questo implica tenere uno **stato condiviso** (ad es. un database in cui il Manager scrive i task e i Worker scrivono i risultati). Dal punto di vista implementativo, potremmo programmare una loop in cui: (a) inviamo a Manager-Claude il goal, (b) lui risponde con una lista di assegnazioni, (c) per ogni assegnazione chiamare un Worker-Claude appropriato, (d) raccogliere i risultati e restituirli al Manager-Claude, che infine fornirà l'output finale combinato. Claude con il suo contesto ampio è utile per mantenere **memoria delle interazioni** tra manager e molti worker. Bisogna però fare attenzione a fornire a Claude (Manager) un prompt che lo istruisca chiaramente a non risolvere da solo i compiti ma a delegare e attendere i risultati – in mancanza di tali istruzioni, un modello potente potrebbe cercare di completare autonomamente il lavoro dei worker invece di coordinarsi.

Implementazione con OpenAI (GPT-4/Codex)

Con GPT-4, possiamo sfruttare la **struttura dei messaggi** (sistema, utente, assistente) per orchestrare ruoli multipli. Ad esempio, il messaggio di sistema potrebbe definire: *"Sei il Manager di un team di agenti AI. Coordini i seguenti specialisti: [lista]. Segui il protocollo: assegna i compiti, attendi i risultati, validali, fornisci output finale."* Dopodiché, si può iniziare passando l'obiettivo. GPT-4 tenderà a elaborare un piano e potenzialmente può anche simulare la risposta di ciascun worker. Un modo efficiente è implementare con codice: la nostra applicazione invia a GPT-4 (Manager) il goal, ottiene la lista di compiti e destinatari. Poi per ogni compito possiamo invocare *separatamente* GPT-4 (o modelli specializzati) con un contesto che includa solo quel compito (e magari la descrizione del worker).

Questo riduce la lunghezza del prompt e permette ad esempio di usare modelli più piccoli/cost-efficient per alcuni sotto-task (Codex per coding, GPT-3.5 per compiti semplici, GPT-4 per i più difficili). Dopo aver ottenuto tutte le risposte dei Worker, si costruisce un messaggio al Manager contenente tutti i risultati, e si chiede a GPT-4 di *validarli e comporli*: *"I worker hanno completato i loro compiti con i seguenti risultati: [...]. Per favore, verificane la correttezza e integra tutto in una soluzione finale."*. GPT-4 a questo punto fornirà il risultato finale, eventualmente segnalando incongruenze. Per la validazione automatica, OpenAI non offre modelli di "critiquing" separati, ma GPT-4 stesso è abile nel ruolo di critico. Potremmo perfino far *rieseguire un worker* se GPT-4 rileva un errore: ad esempio, se il Manager notasse che il risultato del Worker1 è inadeguato, potrebbe richiedere una correzione; il nostro orchestratore allora rifarebbe la chiamata per quel compito con prompt aggiustato. Questo introduce una sorta di **ciclo di feedback** all'interno del pattern manager-worker.

Esempi

Un esempio concreto è un **sistema di assistenti specializzati** in un servizio clienti automatizzato. Il Manager (LLM principale) riceve la richiesta complessa di un utente (es. "Ho problemi col mio ordine, e ho domande sul rimborso"). Il Manager assegna sottocompiti a Worker specializzati: ad esempio un *Worker "Tracking"* per controllare lo stato di spedizione, un *Worker "Policy"* per verificare le politiche di rimborso, un *Worker "Support"* per formulare una risposta empatica. Ciascun agente restituisce la propria parte di informazione. Il Manager poi valida (es. controlla che i dati di tracking siano aggiornati, che la policy citata sia corretta) [17] e combina i risultati in una risposta coerente all'utente. Un altro esempio è un **sistema di analisi aziendale**: il Manager agent riceve l'incarico di analizzare l'andamento di un'azienda, quindi delega a un Worker "Analisi vendite" (che elabora i dati di vendita), a un Worker "Analisi social media" (che analizza sentiment e trend online), e a un Worker "Report" (che assembla testo e grafici). Dopo l'esecuzione parallela, il Manager unisce le analisi in un unico rapporto. In letteratura, pattern gerarchici simili sono stati adottati in *AutoGPT multi-agente* e sistemi come **CAMEL** (dove c'è un "AI manager" che coordina due agenti collaborativi) – dimostrando la capacità di risolvere problemi complessi tramite suddivisione di ruoli.

3. Pattern Critico–Attore (Actor–Critic)

Descrizione

Il pattern Critico–Attore prevede due agenti con **ruoli complementari**: l'**Attore** genera una soluzione o proposta per il compito dato, mentre il **Critico** la **valuta e fornisce feedback** [17] [18]. In pratica, l'Attore è simile all'agente "risolutore" principale, e il Critico funge da "revisore" o "giudice" che passa al setaccio l'output dell'attore individuando errori, inesattezze o violazioni di vincoli (fatti, policy, ecc.) [19] [18]. Questo pattern innesca spesso un ciclo iterativo: dopo la valutazione, l'Attore può **rivisitare la sua soluzione** sulla base delle critiche ricevute, producendo una versione migliorata. Si continua così finché il Critico approva l'output o finché si raggiunge un numero massimo di iterazioni. L'idea ricorda il classico schema *attore-critico* usato nel reinforcement learning, ma qui applicato a generazione di contenuti e ragionamento.

Vantaggi e limiti

Vantaggi: Introduce un livello di **controllo di qualità** interno al sistema. Il Critico funge da filtro che **migliora accuratezza e affidabilità** dell'output, segnalando fatti sbagliati, logica fallace o bias [18] [20]. Ciò riduce il rischio di allucinazioni o risposte scorrette perché ogni output passa una verifica aggiuntiva. Inoltre, separare i ruoli consente di impiegare modelli diversi ottimizzati per ciascun ruolo: ad esempio un modello creativo come Attore e uno rigoroso/factual come Critico. Il feedback iterativo permette al sistema di **apprendere dagli errori** e affinare la soluzione, avvicinandosi a un processo di

revisione "umano" (bozza → revisione → finale).
Limiti: Aggiunge latenza e costo computazionale, dato che **ogni iterazione raddoppia le chiamate** (una dell'attore e una del critico) [21]. Bisogna prevenire loop infiniti di critica-miglioramento: il critico potrebbe continuare a trovare piccole cose da correggere, quindi serve un criterio di arresto (es. un limite di iterazioni o una soglia di qualità raggiunta) [22] [23]. Inoltre, c'è il rischio che il Critico non sia perfetto: se il critico sbaglia valutazione (magari segnalando un errore inesistente o mancando di cogliere un problema), l'Attore potrebbe peggiorare la soluzione o fermarsi prematuramente. Infine, orchestrare la comunicazione tra attore e critico richiede attenzione nel mantenere il contesto: l'Attore deve capire e applicare appropriatamente le critiche.

Implementazione con Claude

Claude, grazie alle sue capacità di **meta-ragionamento** (derivate dall'approccio Constitutional AI), si adatta bene al ruolo di Critico. Un design possibile è utilizzare **due istanze di Claude**: una come Attore e una come Critico. Ad esempio, la prima chiamata a Claude-1 (Attore) genera una risposta iniziale al prompt dell'utente. Poi passiamo questa risposta a Claude-2 (Critico) con un prompt tipo: *"Ecco la risposta proposta: [testo]. Valutala in modo critico, trovando errori fattuali, lacune o violazioni di istruzioni, e suggerisci come migliorarla."* Claude-2 restituirà un elenco di osservazioni e suggerimenti. Quindi rimandiamo a Claude-1 un nuovo prompt contenente la sua risposta precedente e il feedback del Critico, chiedendo di **revisionare la risposta** tenendo conto delle critiche. Questo ciclo può essere implementato iterativamente. Claude tende a seguire molto bene istruzioni di questo genere, essendo addestrato anche a *"auto-riflettere"*. Un'alternativa è usare **un singolo Claude** in modalità "chain-of-thought" per emulare entrambi i ruoli: per esempio, fornirgli un prompt con un formato fisso: prima "**Risposta iniziale:** ...", poi "**Valutazione critica:** ...", poi "**Risposta finale migliorata:** ...". Claude può generare in un colpo solo sia la risposta che la sua auto-valutazione e la revisione. Questa tecnica sfrutta il completamento multiruolo dentro una singola chiamata, ma può essere meno controllabile. Meglio spesso mantenere separati i ruoli in due chiamate per avere più chiarezza. Claude ha anche il vantaggio di un contesto ampio, il che aiuta a includere sia la risposta che il feedback dettagliato nella richiesta di revisione senza troncare informazioni importanti.

Implementazione con OpenAI (GPT-4/Codex)

Con GPT-4 si può implementare creando due agenti distinti: ad esempio usando due conversazioni separate o due *role* fittizi nella stessa conversazione. Un pattern comune è utilizzare **GPT-4 come attore** e magari un'altra istanza (eventualmente GPT-4 stesso o GPT-3.5) come critico. OpenAI non fornisce out-of-the-box un modello critico, ma GPT-4 è già molto capace di analisi critica se opportunamente istruito. Perciò, un flusso manuale sarebbe: prompt utente → **GPT-4 Attore** produce risposta A → prendere A e costruire prompt per **GPT-4 Critico** del tipo: *"Di seguito una risposta fornita da un altro AI. Valutala punto per punto: è corretta e completa? Se trovi errori, spiega quali e come andrebbe corretta."* → GPT-4 Critico restituisce un elenco di valutazioni e suggerimenti. Poi inviamo a GPT-4 Attore un secondo prompt con: *"La tua risposta precedente: [A]. Feedback del critico: [B]. Fornisci una nuova risposta che risolva questi problemi."*. Questo in genere produce una risposta migliorata. Possiamo chiudere qui, oppure iterare di nuovo se necessario (magari facendo ricontrollare al critico). Un accorgimento con GPT-4 è di **mantenere traccia delle versioni**: può essere utile numerare le iterazioni (Risposta v1, Revisione 1, Risposta v2, ...). Il prompt di sistema per GPT-4 può aiutare a definire il tono del critico (ad es. "Sei un critico severo ma costruttivo, la tua valutazione deve essere obiettiva e focalizzata su errori fattuali o logici"). In alternativa all'approccio manuale, esistono implementazioni open source: ad esempio **Self-Refine** e altri script di *prompt engineering* che automatizzano questo scambio attore-critico con GPT. Anche librerie come LangChain forniscono un **CritiqueChain** per prendere un LLM e far generare revisioni sul suo output. Per implementare in Codex contesti di coding: potremmo far scrivere codice all'Attore (Codex model) e poi usare GPT-4 come Critico per eseguire un code review statico, oppure

addirittura eseguire i test e far valutare i risultati (critica basata su test). Ad esempio, l'Attore scrive una funzione, il Critico (GPT-4) legge il codice e commenta errori potenziali, e l'Attore poi corregge.

Esempi

Questo pattern è molto utilizzato in contesti che richiedono **accuratezza elevata**. Un esempio è un *bot per risposte mediche*: l'Attore genera una bozza di risposta al quesito medico di un utente, il Critico (addestrato su linee guida mediche) verifica la correttezza scientifica e l'appropriatezza, suggerendo correzioni se rileva affermazioni dubbie. Solo dopo l'approvazione del Critico la risposta viene data all'utente [17] [18]. Un altro esempio è nella **generazione di codice**: l'Attore (modello tipo Codex) produce uno script in base alle specifiche, il Critico (potrebbe essere GPT-4) rilegge il codice per controllare bug, vulnerabilità o incongruenze con i requisiti e rimanda i suoi commenti. L'Attore quindi fornisce una versione depurata del codice. Questo flusso è simile a come lavorano uno sviluppatore junior (Attore) e un revisore esperto (Critico) in un code review. In ambito accademico, il **pattern di riflessione** (vedi oltre) formalizza il concetto di attore+critico in cui il modello riflette sui propri errori [24] [25]. Ad esempio, il progetto *MathGuru* ha usato un agente Generatore e un agente Riflettore: il Generatore risolve un problema matematico, il Riflettore controlla la correttezza della soluzione e, se trova errori, spiega cosa non va; il Generatore riprova finché il Riflettore conferma che la risposta è giusta [26] [27].

4. Pattern Riflessione Autocritica (Reflection Pattern)

Descrizione

Il pattern di Riflessione introduce un **meccanismo meta-cognitivo** in cui l'agente valuta **criticamente il proprio operato** per migliorarlo iterativamente [28] [29]. A differenza di Critico-Attore, qui tipicamente lo stesso agente (o un gemello dello stesso modello) svolge entrambi i ruoli in sequenza: prima genera una soluzione, poi passa in modalità "osservatore di sé stesso" e **analizza la risposta alla ricerca di errori o mancanze**, quindi la **refina** sulla base delle osservazioni [24] [22]. In pratica, il flusso è: **Generazione iniziale → Auto-valutazione → Miglioramento → (Iterazione)** [24]. Questo pattern mima il processo umano di stendere una bozza, rileggerla criticamente e revisionarla più volte. L'agente può anche incorporare informazioni aggiuntive durante la riflessione (ad esempio recuperando dati esterni se nota un buco informativo).

Vantaggi e limiti

Vantaggi: Consente all'agente di **correggere inconsistenze e approfondire la logica** delle proprie risposte prima di darne una definitiva [30] [31]. Migliora l'accuratezza e la coerenza: errori di calcolo, contraddizioni o affermazioni infondate possono essere individuati dallo stesso modello in fase di riflessione e corretti nella risposta finale [32]. Questo porta a output **più robusti e affidabili**, specialmente in domini dove una singola passata del modello può fallire (matematica, programmazione complessa, ragionamento logico). Inoltre, abilita una forma di **apprendimento iterativo**: l'agente in qualche modo "impara" dalle proprie correzioni all'interno del singolo task, simulando un miglioramento continuo.
Limiti: Richiede **maggiore potenza computazionale** perché ogni iterazione di riflessione aggiunge chiamate al modello e processing extra [33]. Bisogna inoltre progettare con attenzione per **evitare loop indefiniti** di auto-miglioramento: l'agente potrebbe continuare a trovare possibili perfezionamenti senza mai concludere, quindi è utile impostare un limite di iterazioni o criteri di convergenza [33]. C'è anche la sfida del *doppio carico cognitivo*: chiedere a un modello di criticare efficacemente se stesso può essere complesso, soprattutto se la qualità iniziale della risposta è scadente (il modello potrebbe non riconoscere gli errori se non li conosce affatto). In tali casi, la riflessione può degenerare in minimi

aggiustamenti cosmetici senza risolvere i problemi di fondo. Infine, mantenere il contesto di tutto il ragionamento e delle revisioni all'interno dei limiti di token può essere arduo per risposte lunghe o iterative.

Implementazione con Claude

Anthropic ha enfatizzato nelle sue pubblicazioni l'importanza di "modelli che si auto-riflettono", e infatti Claude integra principi di **auto-critiqua** tramite Constitutional AI. Per implementare esplicitamente il pattern, possiamo usare un singolo Claude in due fasi: prima fase generativa e seconda di riflessione. Ad esempio, forniamo la domanda iniziale a Claude e otteniamo la **risposta grezza**. Poi ri-invochiamo Claude con un prompt strutturato: includiamo la *risposta grezza* e istruiamo Claude a **passare in rassegna** quella risposta. Un prompt tipico potrebbe essere: *"Ecco una risposta generata: [testo]. Valuta criticamente questa risposta. Individua eventuali errori fattuali, punti poco chiari o passaggi da approfondire, e spiega come migliorare."*. Claude restituirà un'analisi (spesso enumerando problemi e suggerimenti). A questo punto, facciamo una **terza chiamata a Claude** includendo sia la risposta iniziale che il feedback, e chiedendo: *"Fornisci una nuova risposta che tenga conto delle seguenti correzioni/miglioramenti: [...]"*. Questo generalmente produce una risposta finale di qualità superiore, con errori corretti e dettagli aggiunti. Possiamo iterare più volte se necessario (ad es. due cicli di riflessione). Un altro modo è sfruttare la **strumentazione di LangChain** con Claude: LangChain ha pattern di "Reflection" dove definisci un agent LLM che dopo ogni risposta chiama una funzione di riflessione (che può essere di nuovo un LLM) [34] [35]. Claude può essere integrato in quell'architettura, beneficiando dei suoi 100k token di contesto per rivedere anche conversazioni lunghe. In generale Claude tende ad essere *onesto e critico* sulle proprie risposte se glielo si chiede, spesso riconoscendo errori di ragionamento. È importante formulare la fase di riflessione in modo costruttivo (evitando che Claude "si scusi" semplicemente per ogni potenziale errore invece di correggerlo). Una tecnica utile è dare a Claude una *"checklist"* di aspetti da ricontrollare (esattezza dei fatti, aderenza alla domanda, logica, stile, ecc.) durante la riflessione, in modo da guidare la sua analisi.

Implementazione con OpenAI (GPT-4/Codex)

Con GPT-4, la riflessione può essere implementata in maniera simile: GPT-4 fornisce una prima risposta e poi viene richiesto di **analizzare criticamente** la propria risposta. Possiamo farlo all'interno di un'unica conversazione sfruttando i ruoli. Ad esempio: prompt utente → GPT-4 risponde (Attore). Quindi l'applicazione aggiunge un messaggio utente: *"Analizza la tua risposta sopra. Trova errori o punti migliorabili e descrivi come la correggeresti."*. Possiamo designare questo come uno *step di pensiero interno*: GPT-4 potrebbe fornire una specie di monologo di riflessione. Successivamente, forniamo di nuovo un messaggio utente: *"Ora fornisci una risposta rivista e corretta, tenendo conto della tua analisi."*. Questo spinge GPT-4 a produrre la risposta finale migliorata. Un trucco efficace è usare il formato *Chain-of-Thought invisibile*: OpenAI consente di usare il campo `system` per dare istruzioni di riflessione che non vengono mostrate all'utente finale. Ad esempio, si può mettere nelle istruzioni di sistema qualcosa tipo: *"Prima di dare la risposta finale all'utente, pensa in modo critico alla tua bozza di risposta, e assicurati di correggere eventuali errori."*. Anche se GPT-4 non esplicita la riflessione, questo spesso migliora la qualità (questo è simile a far eseguire una riflessione interna senza dover gestire più turni). Per implementazioni programmatiche più strutturate, ci sono diverse ricerche: **Self-Reflection** (Shinn et al., 2023) e **Reflexion** [36] hanno mostrato che GPT-4 può autovalutare e apprendere da tentativi falliti in compiti complessi come risoluzione di puzzle. Un esempio open source è la *Reflexion* loop: GPT-4 prova a risolvere un problema, se fallisce (magari lo capisce leggendo un test di verifica), allora nella iterazione successiva gli si fornisce anche il "ricordo dell'errore" e un prompt *"Hai fallito perché X. Prova un approccio diverso."*. Questo gradualmente guida GPT-4 a evitare gli stessi errori. Nel coding, Codex e GPT-4 possono generare codice e poi auto-eseguirlo (tramite un wrapper Python) per vedere se

funziona: se fallisce, il log dell'errore viene passato di nuovo al modello come riflessione, che tenta di correggere il codice.

Esempi

Il pattern di riflessione viene applicato in svariati domini. Ad esempio, nei **LLM per matematica**, un agente inizialmente tenta la soluzione di un problema. Poi, in riflessione, ricontrolla il ragionamento passo-passo e magari individua un errore algebrico, lo corregge e rigenera la soluzione giusta [37] [38]. In **campo legale**, un agente può redigere una bozza di contratto, poi rileggerla individuando clausole ambigue o potenziali problemi legali (riflessione), e infine produrre una versione revisionata più solida. Un esempio interessante è l'uso in **sistemi conversazionali**: supponiamo un chatbot dia una risposta che potrebbe violare una policy o essere scortese; un meccanismo di riflessione integrato può far sì che **prima** di inviarla all'utente, il modello la riveda: "La mia risposta contiene qualcosa di inappropriato o errato?" e la editi automaticamente per allinearla alle linee guida. OpenAI ha esplorato idee simili in contesto di *moderation by the model itself*. Sul lato implementativo, **LangChain** offre un esempio denominato "Reflection agent" dove l'agente dopo ogni azione riflette se sta andando fuori strada [39] [40]. Ciò è stato usato per migliorare notevolmente tassi di successo in compiti come giochi di test QA: il modello, quando sbaglia, se ne accorge e ritenta con una strategia diversa, tutto senza intervento umano.

5. Pattern ReAct (Reason + Act)

Descrizione

ReAct è un pattern in cui l'agente combina strettamente **ragionamento e azione** in un ciclo interattivo [41]. Invece di ragionare interamente a "mente chiusa" o di agire senza riflettere, l'agente **alterna fasi di pensiero (Thought)** e **fasi di azione (Act)** [42]. Durante le fasi di pensiero, il modello esegue una sorta di **Chain-of-Thought** esplicita, deducendo quale informazione gli serve o quale passo compiere; nelle fasi di azione, l'agente interagisce con l'ambiente esterno – ad esempio chiamando un tool, effettuando una ricerca, o compiendo un'operazione – e ottiene un'**osservazione (Observation)** in risposta [43] [44]. Queste osservazioni vengono integrate nel passo successivo di ragionamento, e il loop continua finché l'agente non giunge a una **soluzione finale**. In sintesi, ReAct implica che l'LLM pensa a voce alta (trace di ragionamento) e al contempo compie azioni concrete, con ciascun ciclo Thought→Act→Observation→… che guida il successivo [42] [43].

Vantaggi e limiti

Vantaggi: Permette all'agente di essere **dinamico e adattivo**. Integrando azioni (come chiamate a strumenti o query) nel flusso di pensiero, il modello può **colmare lacune di conoscenza in tempo reale** – ad esempio cercando su Wikipedia se gli manca un dato [45] [43]. Il ragionamento intercalato con osservazioni rende l'agente **consapevole del contesto esterno** e capace di aggiustare il piano in base ai risultati ottenuti (feedback loop). Studi hanno dimostrato che ReAct migliora le prestazioni in compiti complessi rispetto a approcci puramente basati su ragionamento interno, e aumenta anche la **trasparenza**: avendo un log passo-passo di pensieri e azioni, è più facile tracciare *come* l'agente è arrivato alla risposta [46]. Supporta la **orchestrazione di tool** in maniera naturale (il prompt stesso prevede che l'agente decida quando invocare quale tool).
Limiti: Senza adeguate cautele, un agente ReAct potrebbe incorrere in **loop infiniti** di pensiero/azione, ad esempio continuando a cercare informazioni anche quando ha già abbastanza elementi per rispondere [47]. Bisogna quindi implementare criteri di terminazione (numero max di step, o logiche che facciano concludere il processo). Inoltre, ReAct **richiede robustezza nella gestione degli errori degli strumenti**: se un'azione fallisce (es. un tool che non risponde o una query senza risultati), l'agente deve

essere in grado di reagire, eventualmente cambiando strategia [48]. Un limite pratico è che il prompt di ReAct (con tracce di pensiero e azioni) può diventare lungo, consumando token di contesto – anche se spesso vale la pena per ottenere maggiore accuratezza. Infine, la qualità dipende dalla capacità del modello di **seguire il format**: ReAct tipicamente richiede prompt con un formato rigoroso (tipo: "Thought: … \n Action: … \n Observation: …"). Modelli meno potenti potrebbero confondersi nel rispettare questa struttura, se non ben istruiti con esempi few-shot.

Implementazione con Claude

Claude può essere utilizzato in stile ReAct fornendo un opportuno prompt che definisce il formato Thought/Action. Ad esempio, si può iniziare la conversazione con un **prompt di sistema** che spiega: *"Devi risolvere il compito alternando pensieri e azioni. Quando pensi, inizia la frase con 'Pensiero:'; quando decidi un'azione, inizia con 'Azione:'; quando ricevi un'osservazione, riportala come 'Osservazione:'."*. Claude tende a seguire istruzioni di formato abbastanza bene. Possiamo poi dargli un input utente (es. una domanda complessa). A quel punto Claude genererà qualcosa come: *"Pensiero: per rispondere devo cercare X. \n Azione: [Cerca(X)]"* (questo è un esempio, l'azione può essere definita come una pseudo-funzione). Il nostro codice esterno deve allora **intercettare** questa azione e svolgerla (eseguire la ricerca X) e fornire a Claude la **Osservazione** risultante (ad esempio il contenuto della pagina trovata). Si invia quindi un nuovo prompt a Claude appending: *"Osservazione: …"* con i dati trovati. Claude continuerà: *"Pensiero: …"* e così via. In pratica, serve un loop dove la logica esterna riconosce quando Claude richiede un'azione (e quale) e fornisce la risposta. Fortunatamente, esistono librerie che facilitano questo: per esempio **LangChain** supporta agenti ReAct con i modelli di Anthropic (basta passare il modello e gli strumenti disponibili e la libreria si occupa dell'orchestrazione). Un dettaglio: bisogna assicurarsi di **limitare le iterazioni**. Con Claude, si può ad esempio dire nel sistema: *"Non effettuare più di 5 cicli di Pensiero/Azione senza arrivare a una conclusione."* Claude solitamente è abbastanza brava a capire quando ha ottenuto ciò che serve per formulare la risposta finale (che in formato ReAct viene spesso indicata con un'azione speciale tipo `Action: Finish[...]`). Claude al momento (v.2) non ha il "function calling" nativo come GPT-4, ma possiamo confidare sul parsing testuale delle richieste di azione. Alternativamente, si potrebbe implementare ReAct in maniera *inner monologue*, dove in realtà Claude esegue il ragionamento e le azioni "invisibilmente" e restituisce solo la risposta finale. Tuttavia questo perde i vantaggi di trasparenza e modularità; meglio sfruttare la piena interattività con environment come progettato.

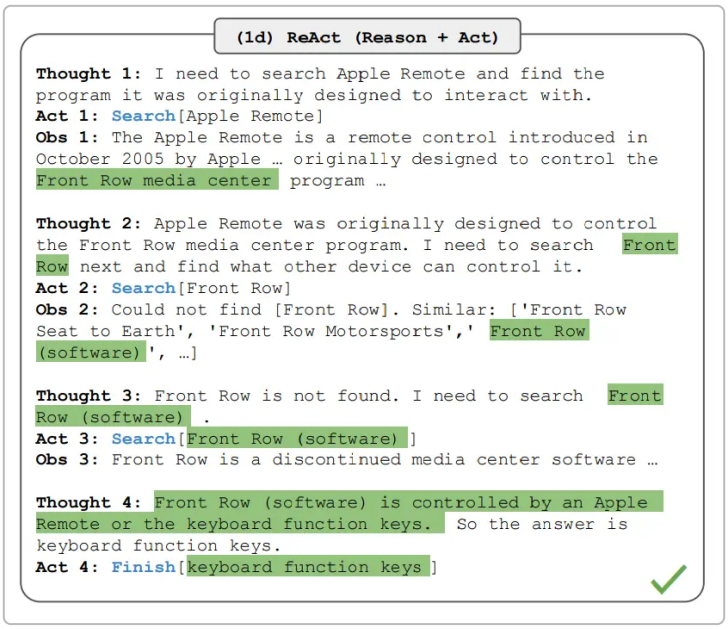
Implementazione con OpenAI (GPT-4/Codex)

OpenAI GPT-4 ha reso più agevole implementare ReAct grazie alle **API con Function Calling**. In pratica, possiamo definire varie *funzioni* (ad esempio `search(query)`, `calculate(formula)` ecc.) e lasciare che GPT-4 le chiami automaticamente durante la conversazione. Questo incarna proprio il concetto di ReAct: GPT-4 ragiona e decide quando invocare una funzione (azione) per ottenere informazioni. Ad esempio, potremmo iniziare con un system message: *"Usa il pensiero step-by-step e chiama funzioni se servono. Non dare la risposta finale finché non hai raccolto tutte le informazioni necessarie."*. Poi un utente chiede: "Chi era il presidente degli USA quando l'URSS si sciolse?". GPT-4 potrebbe pensare internamente e poi emettere un *function_call* a `search("URSS dissoluzione anno")`. La nostra applicazione esegue la ricerca, trova che l'URSS è stata dissolta nel dicembre 1991, passa questa informazione come risposta del tool. GPT-4 la riceve e sa ora la data, quindi potrebbe chiamare un'altra funzione `search("Presidente USA 1991")`, ottiene "George H. W. Bush", poi decide di formulare la risposta finale: "Era George H. W. Bush.". Dietro le quinte, GPT-4 stava facendo ReAct (pensiero implicito + azioni via funzioni). Se non si usa function calling, si può implementare manualmente come con Claude: definendo un formato e parsando l'output. Ad esempio, i prompt few-shot di **Yao et al. (2022)** [49] [43] mostrano come presentare al modello degli esempi in cui c'è un "Thought:" e un'"Action:" e poi un'"Observation:". GPT-4 è molto bravo a imitare questi pattern se glieli

forniamo nel prompt insieme alla nuova domanda. Anche LangChain per OpenAI fornisce out-of-the-box agenti ReAct, sia in versione "text-based" che con function calls. Codex (modello orientato al codice) può essere usato per agenti che interagiscono con un ambiente runtime: ad esempio, si può dare a Codex una "console Python" come strumento; il modello pensa a come risolvere un problema di coding, usa l'azione `python` per eseguire codice, vede l'output e continua finché trova la soluzione corretta. Questo è stato dimostrato in applicazioni come **OpenAI Codex CLI** e in approcci come *ReAct in Python*, dove il LLM potenzia le sue capacità chiamando codice Python su richiesta.

Esempi

Il pattern ReAct è alla base di molti **agent conversationali avanzati**. Un esempio classico: un *agente investigativo* che deve rispondere a una domanda di knowledge (tipo domande stile Trivial Pursuit). Invece di sparare una risposta immediata, l'agente ReAct esegue un ciclo: "Pensiero: la domanda chiede X, forse devo cercare info su Y \n Azione: Search[Y] \n Osservazione: [risultato trovato] \n Pensiero: dal risultato estraggo Z, forse c'è da approfondire \n Azione: Search[Z] \n … \n Azione: Finish[risposta finale]" [50] [51]. In questo modo trova la risposta corretta motivandola con i dati raccolti. Ad esempio, per la domanda *"A parte l'Apple Remote, quali altri dispositivi possono controllare il programma per cui l'Apple Remote fu progettato originariamente?"*, un agente ReAct penserebbe a cercare informazioni su *Apple Remote*, scopre che controlla **Front Row** (programma media center) e poi cerca quali dispositivi possono controllare Front Row, trovando che oltre al telecomando Apple, Front Row può essere controllato da **tastiera e mouse**. Infine fornisce la risposta [52] [53]. Un altro esempio è **Agentic search agents**: dati compiti come "trova le ultime notizie su X e mandamele via email", un agente ReAct potrebbe: usare tool di web search per raccogliere notizie, poi un'azione di email API per inviarle, alternando ragionamento (quali notizie scegliere) e azione (chiamata API). In generale, molti framework attuali (LangChain, LlamaIndex, etc.) implementano agenti ReAct come componente base. Anche il famoso **AutoGPT** (descritto più avanti) al suo interno utilizza un loop di pensare/agire simile a ReAct, ma con in più la pianificazione di lungo termine.



Esempio di traccia ReAct (pensiero + azioni): Ecco uno spezzone dal paper originale di Yao et al. che illustra il processo ReAct. L'agente ragiona in passi (*Thought*) e fa ricerche (*Action: Search[...]*) ricevendo osservazioni dal tool, finché accumula abbastanza informazioni per concludere con un'azione *Finish* che contiene la risposta [52] [53]. Nell'esempio, sta cercando quali dispositivi (oltre all'Apple Remote)

controllano un certo programma (Front Row); vediamo come attraverso più cicli di pensiero/azione giunge alla risposta corretta.

6. Pattern Ciclo AutoGPT (Autonomous Loop)

Descrizione

Il pattern "AutoGPT-style loop" rappresenta un **agente autonomo a ciclo chiuso** che, una volta impostato un obiettivo generale, è in grado di **iterare da solo** pianificando sotto-task, eseguendoli, valutando i risultati e **rielaborando i passi successivi** senza intervento umano [54] [55]. In sostanza, è un meta-pattern che incorpora elementi di Pianificazione, ReAzioni (ReAct), Memoria e Riflessione in un'unica architettura. Il flusso tipico è: l'utente definisce un **goal ad alto livello**, l'agente autonomo genera una lista di compiti necessari (come farebbe un Pianificatore), poi entra in un loop in cui prende il **primo task pendente**, lo esegue (usando strumenti o funzioni se necessario), **analizza il risultato** e l'obiettivo aggiornato, quindi potenzialmente **genera nuovi task** o modifica quelli esistenti, e procede col successivo. Questo ciclo continua finché l'obiettivo principale non è raggiunto o finché l'agente si ferma attendendo input umano [56] [57]. Durante tutto il processo, l'agente mantiene una **memoria a lungo termine** (es. attraverso un vettore store) delle informazioni utili raccolte e uno stato dei task. AutoGPT è l'esempio emblematico di questo pattern: un agente che *si auto-prompta* e si auto-dirige verso il completamento di compiti complessi.

Vantaggi e limiti

Vantaggi: Questo pattern realizza il massimo grado di **autonomia**: l'agente può svolgere compiti multi-step senza richiedere continua supervisione umana [54] [55]. È ideale per obiettivi vaghi o di alto livello che richiedono esplorazione e adattamento continuo (es. "crea un business plan completo per un nuovo prodotto"). L'integrazione nativa di memoria consente all'agente di **imparare dalle iterazioni precedenti** e non dimenticare informazioni chiave man mano che progredisce [58] [57]. Inoltre, il loop autonomo permette di **reagire a ostacoli o risultati imprevisti**: se un sotto-task fallisce o rivela nuove sfide, l'agente può ricalibrare il piano, aggiungere nuovi step, o modificare la strategia. In altre parole, fornisce adattabilità e resilienza – l'agente non è vincolato rigidamente a un piano statico ma lo aggiorna on the fly.
Limiti: D'altro canto, senza vincoli esterni questo pattern può portare a **deriva non controllata**: il rischio maggiore è l'**infinite loop** o l'agente che "perde il focus" esplorando sotto-task irrilevanti [59]. Senza un'adeguata definizione di successo o meccanismi di stop, un AutoGPT può letteralmente andare avanti indefinitamente (sono noti casi in cui consumava migliaia di chiamate senza arrivare a conclusione chiara). Inoltre, dare così tanta libertà a un modello espone ai noti problemi di **hallucination** e decisioni sbagliate amplificati: se l'agente trae una conclusione errata in un'iterazione, quella potrebbe influenzare tutte le azioni successive (effetto valanga) [59]. Ci sono poi **problemi di sicurezza**: un agente auto-operante che esegue codice o naviga potrebbe fare azioni indesiderate se compromesso (es. c'è stato un caso famoso di *AutoGPT che cancellava file importanti a causa di prompt injection*). Per questo, spesso in produzione questi agenti sono confinati con sandbox, limiti temporali e con un umano nel loop per decisioni critiche [60] [61]. Infine, l'approccio è costoso: coinvolge molte iterazioni LLM + chiamate a strumenti, e può richiedere un tuning fine per non sprecare risorse in compiti poco produttivi.

Implementazione con Claude

Implementare un loop stile AutoGPT con Claude significa orchestrare più chiamate in successione e dotare Claude di uno **spazio di memoria persistente**. Possiamo fare leva sul grande contesto di Claude per mantenere molti dettagli nel prompt, ma per una vera memoria a lungo termine conviene usare un

archivio esterno (ad es. un database di vettori per fatti appresi). Un'architettura possibile: definire **un prompt di sistema complesso** per Claude che includa istruzioni come: ruolo = "Project Assistant AI", obiettivo generale = X, formato per tenere traccia del *Task List*, *Risorse Conosciute*, *Piano Corrente*, ecc. Ad esempio, potremmo avere nel prompt qualcosa come:

```
Stato Attuale:
- Obiettivo: <descritto dall'utente>
- Task in coda: 1) ..., 2) ...
- Risorse raccolte: ...
Linea di Azione:
Devi prendere il primo task, svolgerlo e aggiornare lo stato.
Se hai bisogno di usare strumenti, descrivi l'azione.
...
```

In pratica, diamo a Claude una struttura per ragionare sul *cosa fare dopo*. Claude può quindi restituire sia l'azione intrapresa che un nuovo elenco di task aggiornati. Ad esempio: *"Azione: cerco informazioni su …; Aggiornamento Task: aggiunto un nuovo task per analizzare i dati trovati; Commento: risultato della ricerca memorizzato."*. Il nostro orchestratore esegue l'azione (es. chiama API o tool) e poi richiama Claude fornendo l'osservazione e lo stato aggiornato (compresi i task rimasti). Claude itererà. In pratica, scrivere un prompt di stato robusto è la parte più difficile, perché deve includere abbastanza contesto per consentire a Claude di ragionare, ma non così tanto da fargli superare i limiti di token o confonderlo. Fortunatamente, la finestra di Claude (100k token in v2) è ampia e ciò lo rende ottimo per agenti autonomi estesi: possiamo "incollare" gran parte della knowledge acquisita nel prompt ad ogni iterazione. Un altro approccio modulare è usare **LangChain con OpenAI** per la logica generica e sostituire GPT-4 con Claude come LLM. Ad esempio, esiste *BabyAGI* (un progenitore di AutoGPT) implementato in Python, in cui basta sostituire le chiamate OpenAI con chiamate a Claude. Bisogna implementare manualmente i controlli di sicurezza: ad esempio, aggiungere nel prompt di sistema di Claude delle *regole* (tipo *"non eseguire comandi distruttivi sul filesystem"*, *"non uscire dal tuo ruolo"*, ecc.) e mettere eventualmente un time-out sul numero di iterazioni. Claude, essendo addestrato con Constitutional AI, potrebbe di suo evitare azioni palesemente maliziose, ma per sicurezza va trattato come un agente potenzialmente *super creativo* – quindi con sandbox se interagisce col sistema (es. limitare l'accesso a certe funzioni).

Implementazione con OpenAI (GPT-4/Codex)

AutoGPT è nato originariamente proprio con GPT-4, quindi seguiremo quell'esempio. L'implementazione prevede un **ciclo principale** in un programma Python:

1. **Inizializzazione:** definire l'obiettivo e magari un primo elenco di passi (GPT-4 può generarlo o possiamo averne uno statico predefinito).
2. **Loop:** inviare a GPT-4 un prompt che include contesto attuale (stato dei task, risultati finora, riflessioni precedenti) e chiedere *"Decidi il prossimo passo da compiere."*. GPT-4 risponderà con un'azione o con la conclusione del lavoro.
3. **Esecuzione Azione:** Se GPT-4 ha richiesto di usare un tool (es. via function calling o in testo), l'app esterna esegue quell'azione. Ciò può comportare chiamate API (web search, calcolo, ecc.), interazioni col filesystem (AutoGPT ad esempio poteva generare e leggere file), ecc. Ottenuto il risultato, lo aggiungiamo alla memoria.
4. **Valutazione & Pianificazione:** Chiamare di nuovo GPT-4 fornendo l'esito dell'azione, chiedendo se l'obiettivo è raggiunto o in caso contrario quali sono i nuovi sotto-task. GPT-4 aggiornerà la

lista di task (può aggiungerne di nuovi in base a quanto scoperto) e fornirà eventualmente una *"ponderazione"* (ad esempio: *"Il risultato X ottenuto suggerisce di fare Y come prossimo passo."*).
5. Ripetere il loop.

Durante questo processo, GPT-4 funge da **motore decisionale centrale**. Possiamo anche affiancare un modello meno costoso per memorizzare e gestire la knowledge: ad esempio usare embedding e un vettore DB. Ogni volta che GPT-4 genera informazioni nuove importanti, le inseriamo in un index; e ogni volta che costruiamo il prompt, facciamo una query su quell'index per recuperare i dati pertinenti da ricordare (questo aiuta a non sforare la finestra di contesto). OpenAI GPT-4 con function calling può direttamente orchestrare alcune cose: possiamo definire funzioni per "aggiungiTask(nome)" o "segnaCompleto(task)" e GPT-4 le chiamerà per manipolare una lista di task tenuta dal codice. Tuttavia, spesso conviene una gestione esplicita nel codice per avere più controllo. Anche con GPT-4 bisogna inserire misure di **sicurezza e guardrail**: OpenAI ha delle policy e spesso GPT-4 rifiuta di eseguire azioni pericolose, ma per scrupolo è bene, ad esempio, evitare di dare all'agente accesso completo al sistema operativo; se serve manipolare file, farlo in una directory isolata; se fa web requests, filtrarle. Per evitare loop infiniti, si può impostare un contatore max di iterazioni dopo cui l'agente deve chiedere conferma umana. AutoGPT originale infatti spesso chiedeva *"Posso continuare? [yes/no]"* dopo tot passi. In un contesto Codex, è possibile creare agenti auto-iteranti per coding (che scrivono codice, eseguono test, correggono errori, ecc.). In generale, la chiave è che GPT-4 deve essere istruito chiaramente a: generare piani, prendere decisioni su cosa fare dopo, e sapere quando fermarsi (ad esempio quando il task list è vuoto o l'obiettivo soddisfatto).

Esempi

Un esempio famoso di questo pattern è proprio **Auto-GPT**: si chiedeva all'agente di porsi un obiettivo come *"avviare un'impresa di e-commerce"* e lui avrebbe generato sotto-obiettivi (ricerca di mercato, trovare prodotti, creare sito, marketing), portando avanti ciascuno iterativamente [56] [57]. Nella pratica, AutoGPT ha mostrato la capacità di scrivere codice da solo, leggere documentazione, migliorare il proprio codice, fare query e analizzare risultati – tutto senza intervento umano, salvo la configurazione iniziale. Un altro esempio è **BabyAGI**, dove l'agente prende un obiettivo e continuamente aggiorna una "to-do list" di compiti da eseguire (prioritizzandoli in base allo stato corrente). Ad esempio, se l'obiettivo è "scrivere un articolo e pubblicarlo sui social", l'agente può suddividere in: ricerca info, stesura articolo, revisione, creazione immagine, posting. Mentre esegue "ricerca info" potrebbe scoprire nuove domande da approfondire e aggiungerle alla lista. In campo produttivo, al momento agenti del genere sono spesso tenuti con **human-in-the-loop** per sicurezza [60], ma esistono prototipi interessanti: ad esempio un agente finanziario che monitori costantemente il mercato e aggiusti un portafoglio di investimento (l'utente dà solo l'obiettivo "massimizza il rendimento mantenendo rischio moderato", e l'agente fa cicli di analisi dati -> trading -> valutazione performance -> nuovo trade) [62]. Questi agenti autonomi sono una frontiera entusiasmante, ma come evidenziato, richiedono robusti meccanismi di controllo per evitare comportamenti imprevisti.

7. Pattern Albero del Pensiero (Tree-of-Thought, ToT)

Descrizione

Il pattern **Tree-of-Thought (ToT)** estende il normale ragionamento lineare in una **esplorazione ad albero** di possibili soluzioni [63]. Invece di seguire un singolo filo logico come nella Chain-of-Thought, l'agente genera **ramificazioni multiple di pensiero**: in pratica esplora diverse idee, piani o parziali soluzioni in parallelo (o pseudo-parallelo) come rami di un albero di ricerca [64]. Ad ogni "nodo" (stato intermedio del ragionamento), l'agente può espandere vari possibili passi successivi, creando diramazioni. Successivamente, queste traiettorie vengono **valutate e potate**: i rami poco promettenti

vengono scartati e l'agente converge sul percorso migliore o su una combinazione dei migliori [64] [65]. L'output finale deriva quindi dal ramo dell'albero giudicato ottimale o da una sintesi dei rami più fruttuosi. Questo ricorda algoritmi come la **branch & bound** o la ricerca in ampiezza/profondità con backtracking, applicati però al ragionamento di un LLM.

Vantaggi e limiti

Vantaggi: Promuove la **creatività e completezza** del ragionamento. Esplorando più percorsi, l'agente ha meno probabilità di incappare in un *vicolo cieco* o in un optimum locale: se una prima idea non funziona, c'è già un'alternativa pronta [66]. Ciò aumenta le chance di trovare soluzioni corrette a problemi difficili (p.es. puzzle, giochi, problemi logici complessi) dove un singolo chain-of-thought lineare potrebbe fallire. Il ToT pattern aiuta anche a **evitare bias iniziali**: l'agente considera prospettive multiple prima di decidere, un po' come farebbero più persone che pensano indipendentemente e poi confrontano le soluzioni. In alcuni studi, questo approccio ha mostrato di migliorare i risultati in compiti come pianificazione e gioco (ad es. risolvere il Cubo di Rubik mentalmente) rispetto a CoT lineare. Inoltre, offre un grado di **parallelismo concettuale**: volendo, i rami potrebbero essere esplorati in parallelo su più istanze di modello, riducendo i tempi per problemi che lo consentono.
Limiti: Il principale svantaggio è il **costo computazionale** elevato [67]. Generare e valutare molteplici ramificazioni richiede parecchie chiamate al modello (o una chiamata molto grande che contenga l'intero albero ragionato). Ad esempio, se per ogni passo si considerano 3 opzioni e si fanno 4 passi, si valutano fino a 3^4 = 81 percorsi: uno sforzo decisamente superiore rispetto a uno singolo. Serve inoltre definire una **euristica di valutazione** dei rami: il modello stesso può stimare quale ramo sembra più promettente (introducendo però la possibilità di errore di valutazione), oppure si può definire una funzione esterna (ad es. un test di validità logica). C'è poi complessità implementativa nel fare backtracking: bisogna gestire come passare il contesto di un ramo specifico al modello per espanderlo ulteriormente senza interferenze da altri rami. Un altro limite è che non tutti i problemi beneficiano di un approccio multi-ramo: per questioni semplici, può essere overkill e portare persino confusione. Infine, la valutazione dei rami non garantisce di scegliere quello perfetto: se la valutazione è imperfetta, l'agente potrebbe comunque scartare la strada giusta o tenere quella sbagliata (questo è analogo al problema dell'esplorazione vs sfruttamento).

Implementazione con Claude

Per implementare Tree-of-Thought con Claude, possiamo usare la tecnica di **multi-prompting**. Un modo semplice è iterativo: poniamo un problema a Claude e chiediamogli di **generare diverse possibili soluzioni o passi successivi**. Ad esempio: *"Elenca 3 possibili approcci per risolvere questo problema e i passi conseguenti per ciascuno."*. Claude restituirà qualcosa come: **Opzione A:** ..., **Opzione B:** ..., **Opzione C:** ... con i rispettivi ragionamenti. A questo punto, dobbiamo **valutare** questi percorsi. Possiamo chiedere a Claude stesso di valutarli: *"Valuta i tre approcci proposti considerando accuratezza e fattibilità, e scegli il migliore motivando la scelta."*. Claude allora fornirà un giudizio, magari scegliendo l'Opzione B come la più promettente. Possiamo allora dire a Claude: *"Bene, adotta l'approccio B e continua il ragionamento in dettaglio passo-passo."*. A quel punto Claude prosegue su quel ramo prescelto. Se lungo questo ramo emergono altre diramazioni (ad esempio un bivio decisionale), possiamo ripetere: far generare a Claude le alternative, farle valutare e scegliere. In alternativa, potremmo far generare i rami iniziali in parallelo usando più istanze: e.g., tre chiamate separate a Claude ciascuna con un prompt leggermente diverso per esplorare un'idea differente. Poi prendere i 3 output e concatenarli in un prompt a Claude dove funge da *valutatore*. Questo utilizza Claude sia come generatore di ramificazioni che come critico/valutatore. Si può anche coinvolgere una metrica esterna: ad esempio, per un puzzle con soluzione nota, potremmo far eseguire un check esterno (una funzione) su ciascuna soluzione proposta e vedere quale è corretta. Claude può gestire un albero di pensieri di medie dimensioni grazie al lungo contesto, ma bisogna fare attenzione a guidarlo chiaramente: una difficoltà è far sì che

mantenga separate le branche senza mescolarle. Dare formattazione chiara (liste numerate, titoli per opzione) aiuta. Con il framework **LangChain**, esiste una implementazione chiamata *"HYDE + Tree-of-thought"* dove l'LLM genera possibili soluzioni e poi verifica. Claude può essere usato in quell'architettura perché è molto capace nel seguire istruzioni di generazione di più output distinti. Tenere il chain-of-thought **esplicito nel prompt** (tipo: "Pensiero 1: ..., Pensiero 2: ...") può aiutare a successivamente riferirsi ai diversi rami.

Implementazione con OpenAI (GPT-4/Codex)

GPT-4, data la sua forza nel ragionamento, è molto adatto a Tree-of-Thought. Un metodo consiste nell'uso di un **prompt tree**: ad esempio, c'è un progetto open-source chiamato "Tree of Thoughts" in cui definisci una funzione `think(depth)` che chiama GPT-4 per generare possibili pensieri fino a una certa profondità, e una funzione `evaluate(states)` che chiama GPT-4 per valutare uno stato o decidere quale ramo continuare. Possiamo replicare questo:

- **Generazione rami:** Forniamo a GPT-4 un prompt: *"Stai risolvendo il problema X. Fornisci 3 possibili linee di ragionamento distinte, numerate, ciascuna con il primo passo."*. GPT-4 restituirà 3 opzioni di passo 1.
- **Espansione:** Per ogni opzione, prendiamo quel passo e chiediamo (in parallelo o in serie): *"Dato questo inizio di soluzione: [passo1], come prosegue? Fornisci il passo 2."*. Ciò espande l'albero in ampiezza o profondità a seconda di come vogliamo procedere (possiamo fare DFS o BFS generando alcuni livelli per ogni ramo).
- **Valutazione:** A intervalli regolari di profondità, chiamare GPT-4 con il contesto di un ramo completo e chiedere: *"Valuta quanto è vicino alla soluzione o promettente questo ragionamento (dà un punteggio da 1 a 10)."*. GPT-4 darà un punteggio o un giudizio.
- **Potatura/selezione:** Manteniamo i migliori k rami (in base al punteggio) e potiamo gli altri. Poi continuiamo ad espandere quelli selezionati.
- **Termine:** Quando uno dei rami raggiunge un criterio di soluzione (es. formula risolta, domanda risolta) o abbiamo raggiunto una profondità massima, ci fermiamo. Possiamo eventualmente far fare a GPT-4 un ultimo confronto tra le soluzioni finali rimaste e chiedere di sceglierne una definitiva.

Una variazione meno tecnica: potremmo istruire GPT-4 direttamente a generare un albero in una sola chiamata. Per esempio: *"Considera diverse possibili soluzioni. Fai un ragionamento esplorando alternative e contrassegnale come (a), (b), (c) e discuti i pro e contro di ciascuna, poi scegli la migliore."*. In un'unica risposta GPT-4 potrebbe presentare: (a) soluzione X, (b) soluzione Y, (c) soluzione Z, con commenti, e infine dire "tra queste, la migliore è X perché...". Questo è un *mini tree-of-thought* racchiuso in un output, e spesso GPT-4 già fa qualcosa di simile quando gli chiedi "valuta diverse opzioni". Non è vero e proprio branching search, ma concettualmente simile. Comunque, implementazioni come il paper "Tree of Thoughts (2023)" di Shyam et al. forniscono un framework completo, e GPT-4 fu uno dei modelli testati. Se volessimo usare Codex, potremmo pensare al gioco della programmazione: generare varie implementazioni candidate di un algoritmo e poi testarle con alcuni input, scartando quelle che falliscono (i test fungono da valutazione). Ciò sarebbe un ToT guidato da codice.

Esempi

Un esempio didattico: risolvere l'enigma **"Missionari e cannibali"** con un agente ToT. Il problema richiede pianificazione con tentativi ed errori: un agente GPT con Tree-of-Thought potrebbe esplorare più sequenze di mosse (rami), backtrackare quando vede che un ramo porta a uno stato invalido (cannibali > missionari) e infine trovare la sequenza corretta. Senza l'esplorazione ramificata, un LLM potrebbe fermarsi al primo tentativo errato pensando sia impossibile, mentre con ToT può considerare

alternative. Altri esempi: **rompicapi logici o matematici complessi**, dove ci sono varie strade da provare (pensiamo a dimostrare un teorema: il modello potrebbe provare approcci diversi – caso 1, caso 2, ecc. – e poi capire quale fila). Nel **gaming AI**, questo pattern è come far fare al LLM il lavoro di un algoritmo di ricerca giochi: ad esempio nel *gioco 2048*, un LLM ToT genera sequenze di mosse e valuta il punteggio raggiunto, scegliendo la migliore sequenza. Un caso reale documentato è l'uso di *Monte Carlo Tree Search con LLM* (tecnica LATS, Language Agent Tree Search): qui l'LLM (ad esempio GPT-4) simula partite o soluzioni multiple e usa un criterio di ricompensa per convergere [68] [69]. In definitiva, Tree-of-Thought è un pattern potente per situazioni in cui **una singola linea di ragionamento potrebbe non essere sufficiente** e serve valutare opzioni alternative prima di decidere.

8. Pattern Agente con Strumenti e Conoscenza (RAG)

Descrizione

Questo pattern – spesso chiamato **Tool-Use** o **Retrieval-Augmented Generation (RAG)** – consiste nel dotare l'agente di **accesso dinamico a strumenti esterni o basi di conoscenza**, integrando strettamente queste chiamate nel processo di generazione [70] [71]. In pratica, l'agente può, quando necessario, invocare API, eseguire funzioni, consultare database o motori di ricerca, e poi **incorporare i risultati** ottenuti nel proprio ragionamento e nelle risposte [70] [71]. Un caso particolare molto diffuso è RAG: l'agente interroga un **sistema di ricerca o vettore di documenti** per recuperare documenti rilevanti, e utilizza quei documenti per generare una risposta accurata [72] [73]. Più in generale, il pattern strumento include calcolatrici, servizi esterni (meteo, mappe), capacità di eseguire codice, manipolare file, ecc. L'orchestrazione può essere scritta a mano (nel prompt, come visto in ReAct, decidendo manualmente quando chiamare i tool) oppure automatizzata con meccanismi come il *function calling*.

Vantaggi e limiti

Vantaggi: Estende significativamente le capacità dell'agente oltre il puro testo: può **ottenere informazioni aggiornate** non presenti nel suo modello (ad esempio notizie correnti, dati aziendali privati) [74] [75], può effettuare **calcoli precisi** (superando la tendenza dei LLM a sbagliare i conti) e può svolgere azioni nel mondo esterno (come inviare email, modificare un calendario, ecc.). Ciò rende l'agente molto più utile e riduce le **allucinazioni** perché può verificare i fatti in tempo reale: piuttosto che inventare una risposta, può richiamare l'informazione da una fonte affidabile [74]. Nel caso di RAG, fondera i benefici del search con quelli del generative: il modello genera testo guidato da contenuti effettivamente reperiti, migliorando accuratezza e *faithfulness*. Un altro vantaggio è la modularità: possiamo aggiungere nuovi tool all'agente man mano, rendendolo **componibile** (ad es. aggiungiamo un traduttore, un calcolatore, ecc. e il modello li userà quando opportuno).
Limiti: La sfida principale è l'**integrazione corretta** tra modello e strumenti. Se l'**interfaccia (schema) del tool cambia o è inaspettata**, l'agente potrebbe fornire input errati e **rompere il flusso** [76]. Ad esempio, se un'API restituisce un formato JSON complicato, il modello potrebbe non riuscire a leggerlo senza un parsing adeguato. Inoltre, c'è latenza aggiuntiva per via delle chiamate esterne e potenziali errori di rete o time-out che l'agente deve saper gestire. Dal punto di vista della progettazione dei prompt, dare all'agente troppi strumenti può confonderlo: deve capire *quando* è necessario usarli e quando no – modelli meno sofisticati potrebbero chiamare strumenti inutilmente o al contrario ignorarli quando servono. Serve spesso un po' di *fine-tuning* o di contesto esempio per far emergere la corretta utilizzo dei tool. Un altro limite nel contesto RAG è che l'agente dipende dalla qualità del motore di ricerca e dei documenti: se i documenti di knowledge sono rumorosi o insufficienti, l'agente può comunque allucinare colmate le lacune. In sintesi, l'agente con tool è potente ma aggiunge uno **strato di complessità**: oltre a ragionare sul problema, deve anche decidere la strategia di interrogazione di tool esterni.

Implementazione con Claude

Claude non offre (almeno al 2025) un meccanismo nativo di function calling come GPT-4, ma nulla vieta di implementarlo via prompt. Possiamo adottare la stessa strategia vista in ReAct: definire un formato di output dove Claude, quando ritiene opportuno, emette ad esempio: *"[TOOL: nome_tool, parametro]"*. All'inizio della conversazione, forniamo in system prompt l'elenco dei tool disponibili e il formato di chiamata. Ad esempio: *"Hai a disposizione i seguenti strumenti: 1)* `WebSearch(query)` *per cercare sul web, 2)* `Calculator(expr)` *per calcoli, 3)* `DBQuery(sql)` *per interrogare il database. Usa il formato [TOOL: ...] quando vuoi invocarli."*. Se Claude segue queste istruzioni, il nostro codice wrapper può catturare quell'output speciale, eseguire l'azione corrispondente e restituire a Claude il risultato (magari formattandolo come *"[RISULTATO: ...]"*). Questo è molto simile al pattern ReAct e, di fatto, ReAct con tool è praticamente uguale al pattern in questione. La differenza è che qui ci concentriamo sull'integrazione tool/knowledge, che può essere anche in *singola iterazione*: ad esempio, se l'utente chiede "Dammi l'ultimo prezzo delle azioni Tesla", l'agente direttamente fa [TOOL: WebSearch("TSLA stock price")] e con l'osservazione risponde, senza troppi passi di riflessione. Con Claude è utile anche sfruttare la **sua finestra di contesto per RAG**: possiamo integrarlo con un sistema di retrieval in questo modo – la domanda utente viene usata per fare query su un **Vector Store** (fuori da Claude), otteniamo ad es. i 3 documenti più rilevanti, e li forniamo a Claude in prompt insieme alla domanda, dicendo: *"Utilizza i seguenti documenti per rispondere:"*. Claude mescolerà le informazioni fornite nei documenti con la sua capacità generativa, producendo tipicamente una risposta arricchita di dati estratti (magari anche con citazioni se glielo chiediamo). Questo approccio RAG è molto usato con LLM ed è facilmente realizzabile con Claude via API. Per esempio, la catena sarebbe: query utente → cerca in knowledge base (es. con strumenti tipo LlamaIndex o tramite embedding + ricerca simile) → prendere i testi trovati → promptare Claude: *"Documenti trovati: [doc1] [doc2] [doc3]. Domanda: [utente]. Rispondi usando le info date."*. Claude restituirà una risposta con i contenuti corretti quasi fosse *open-book*. Un dettaglio: meglio istruire Claude a **non inventare se non presente nei documenti** (lo si può dire esplicitamente: *"Se l'informazione non è nei documenti sopra, dì che non lo trovi."*). In generale, con Claude si può realizzare tutta la panoplia di tool: integrarlo con Python (es. c'è un wrapper che permette a Claude di dare comandi Python, simile a Replit-Alpha), con servizi web, ecc., ma serve il solito controllo esterno, niente di automatico integrato. Anthropic inoltre pubblica best practice su come *"chain-of-thought with tools"* che possono essere sfruttate; per esempio, suggeriscono di mettere sempre i risultati dei tool nel prompt successivo così che Claude possa usarli direttamente.

Implementazione con OpenAI (GPT-4/Codex)

L'implementazione è notevolmente semplificata dal meccanismo di **Function Calling** introdotto da OpenAI. In pratica, si definiscono in fase di API call le funzioni disponibili con la loro "firma" (nome, parametri). Quando GPT-4 vede che per rispondere sarebbe utile una funzione, invece di rispondere all'utente produce un `function_call` JSON con il nome e gli argomenti. Il developer riceve questo output, esegue la funzione corrispondente nel proprio ambiente, e poi invia a GPT-4 un nuovo messaggio del *function response* con il risultato. GPT-4 a quel punto incorpora il risultato nel contesto e fornisce la risposta finale all'utente [77] [78]. Ad esempio, configuriamo la funzione `search_news(query)`; se l'utente chiede *"Chi ha vinto l'ultima Champions League?"*, GPT-4 potrebbe fare `function_call: search_news("ultima Champions League vincitore")`. Noi facciamo scraping e restituiamo: "Il Manchester City ha vinto l'ultima Champions League (2023)". GPT-4 riceve questa stringa come risultato e risponde all'utente: *"L'ultima Champions (2023) è stata vinta dal Manchester City."*. Tutto avviene in modo fluido. Questa capacità rende la costruzione di agenti con tool **molto più robusta**, perché il modello stesso decide quando e come chiamare, e ha la garanzia di ottenere l'output in un formato strutturato (non deve parsare testo). Oltre alle funzioni custom, GPT-4 integrato in plugin può accedere ad API come navigazione web, database di knowledge (ad es. c'è un plugin per Wikipedia, uno per WolframAlpha per i calcoli, ecc.). Senza function calling, si potrebbe

replicare come con Claude ma GPT-4 era incline a produrre output testuali mescolati, quindi era meno affidabile. Per RAG, il tipico flusso con OpenAI è: fare una query di embedding su un vector store per la domanda utente, ottenere i documenti top-k, quindi incollarli nel prompt di GPT-4 (magari come `context` in system message). OpenAI consiglia di delimitare bene questi documenti (ad es. con delimitatori ``` o con markdown quote) e poi istruire GPT-4: *"Usa queste fonti per rispondere. Se mancano info, dillo."*. GPT-4 essendo molto potente nella comprensione testuale, estrarrà i dettagli pertinenti e li includerà. Se vogliamo spingerci oltre, potremmo far citare le fonti: ad esempio, dire a GPT-4 di restituire frasi con riferimento (1, 2, 3 corrispondenti ai documenti dati). Codex in particolare si usava molto con tool come l'esecuzione del codice Python per fare calcoli. Ad esempio **GPT-Engineer** e progetti simili facevano scrivere codice a GPT-4/Codex e poi lo eseguivano, riportando il risultato (compilazione o output) al modello per correggere errori, un po' come un tool di verifica runtime.

Esempi

Questo pattern è ormai onnipresente: qualsiasi **ChatGPT plugin** è un esempio di agente con tool. Ad esempio, il plugin di OpenTable permette al modello di fare ricerche di ristoranti e prenotazioni: l'utente chiede "trova un ristorante italiano a Napoli stasera per 4 persone", l'agente chiama l'API di OpenTable, riceve le opzioni e poi risponde con i risultati. Un altro esempio comune: agenti per **question answering su documenti aziendali** – qui l'agente utilizza il pattern RAG. Poniamo l'utente chieda *"Qual è l'ultima politica ferie dell'azienda?"*; l'agente effettua una ricerca sul knowledge base interno (documenti HR) e trova il PDF con la policy aggiornata, estrae il punto rilevante e lo include fedelmente nella risposta [72] [73]. Questo garantisce che la risposta sia basata su contenuto reale piuttosto che su allucinazioni o policy obsolete memorizzate. Nel campo coding, c'è **GitHub Copilot**: è principalmente un LLM puro, ma in VSCode è integrato con l'IDE quindi funge quasi da agente con tool – ad esempio può accedere alla base di codice (contesto), e quando genera suggerimenti può sfruttare l'esecuzione dei test (in Copilot Labs esiste una funzione in cui l'AI esegue i test e prova a correggere il codice se falliscono). Un esempio di ricerca accademica è **WebGPT** di OpenAI: un GPT-3 modificato per fare browsing web, leggere risultati e sintetizzare risposte con citazioni. Anche se non era chiamato RAG all'epoca, di fatto implementava questo pattern (LLM + strumento di ricerca) per migliorare accuratezza. In generale, *ogni volta che vediamo un LLM consultare qualcosa o usare calcolatrici*, è il pattern di tool use in azione, che rende gli agenti molto più pratici per compiti realistici.

9. Pattern Catena di Prompt (Prompt Chaining)

Descrizione

Il pattern della Catena di Prompt prevede di suddividere l'interazione LLM in **più passaggi sequenziali**, dove **l'output di un passo diventa l'input del passo successivo**, allo scopo di raffinare gradualmente la risposta [79]. Invece di porre una singola domanda complessa al modello, si crea un flusso di lavoro strutturato: ad esempio, primo prompt per generare un'idea grezza, secondo prompt per svilupparla, terzo per verificarla, e così via. Ogni *anello* della catena è progettato per **affinare o aggiungere un aspetto** alla risposta. Questo pattern è concettualmente semplice ma molto potente: consente di incorporare logica deterministica o controlli tra un prompt e l'altro, e di guidare il modello a un risultato finale attraverso passaggi intermedi chiari.

Vantaggi e limiti

Vantaggi: La suddivisione in passi **riduce la complessità cognitiva per il modello** in ogni singola query, spesso migliorando la qualità. Il modello può concentrarsi su un sotto-task alla volta (es. prima generare un elenco di punti chiave, poi espandere ciascun punto) invece di fare tutto in un colpo solo. Ciò rende il processo più **controllabile e interpretabile**: possiamo ispezionare gli output intermedi e

individuare dove qualcosa va storto. Il chaining permette inoltre di **inserire logica personalizzata** tra i prompt: ad esempio, dopo il primo output possiamo applicare un filtro o trasformazione (anche con codice) prima di passare al prossimo prompt. Si possono anche combinare modelli diversi in pipeline: magari un modello genera testo, un altro lo traduce, un altro lo verifica grammaticalmente. In sintesi, il prompt chaining è flessibile e componibile, favorisce il riutilizzo di prompt specializzati e aiuta a ottenere risposte finali più coerenti perché si è costruita passo passo [79].
Limiti: Ogni anello aggiunge overhead (più chiamate, più latenza). Inoltre, errori in un passo possono propagarsi nei successivi: se il primo prompt genera un elenco di punti non pertinenti, i passi successivi elaboreranno comunque quelli (spazzatura in ingresso, spazzatura in uscita). Quindi è fondamentale curare molto i prompt iniziali o inserire verifiche. A differenza di un singolo prompt, c'è un po' più di progettazione necessaria: bisogna decidere *come* suddividere il compito ottimamente – farlo male potrebbe peggiorare il risultato. Un rischio è anche la **perdita di contesto**: se ogni passaggio non reintroduce adeguatamente il contesto importante, il modello potrebbe "dimenticare" requisiti iniziali. Va gestito passando sempre ciò che serve (es. le istruzioni originali dell'utente) lungo la catena. Infine, c'è una certa soglia oltre la quale troppi passaggi complicano inutilmente: bisogna saper trovare il giusto numero di step (non troppo pochi da lasciare il lavoro sporco al modello, non troppi da diluirlo e accumulare errori di arrotondamento del contesto).

Implementazione con Claude

Prompt chaining con Claude può essere fatto in modo *inline* o *outline*. Metodo inline: mantenere un'unica conversazione e guidare Claude attraverso passi con messaggi di sistema/utente. Ad esempio, primo messaggio utente: *"Step 1: analizza la domanda e forniscimi un elenco puntato di aspetti da coprire."* -> Claude risponde con i punti. Poi secondo messaggio: *"Step 2: per ciascun punto fornisci un paragrafo di spiegazione."* -> Claude produce paragrafi. In pratica, l'utente (o il programma orchestratore) funge da regista indicando a Claude fase per fase cosa fare. Dato che Claude non ha un rigido rispetto ruoli come GPT, questo di solito funziona bene con un opportuno contesto. Metodo outline: fare chiamate separate. Ad esempio, chiamata 1: prompt con istruzioni Step1, ottieni output A; chiamata 2: prompt con istruzioni Step2 + includi l'output A come contesto, e così via. Il vantaggio di chiamate separate è che possiamo intervenire programmaticamente: es. tra step1 e step2 possiamo riordinare i punti se serve, oppure filtrare. Claude gestisce molto testo, quindi includere l'output precedente come "ecco cosa hai prodotto finora, ora continua" non è un problema. Un caso interessante è la **rifinitura dello stile**: potremmo fare Step1 con Claude v2 (che è molto bravo tecnicamente), poi Step2 con Claude Instant (più veloce ed economico) per parafrasare in stile più colloquiale, per esempio. L'**implementazione con LangChain** di catene è diretta: definisci i prompt in serie. Possiamo configurare in JSON/YAML una *SequentialChain* con Anthropic LLM come step. Un esempio di catena: Input utente -> Claude (fase 1: genera bozza) -> funzione Python (corregge errori evidenti o aggiunge dati provenienti da DB) -> Claude (fase 2: revisiona e rifinisce la bozza integrando i dati). La creatività sta nel come orchestrare i passaggi. Claude può essere usato anche per travasare conoscenza tra fasi: ad esempio, potremmo fargli generare uno *structured output* in uno step (magari JSON con campi), e nel passo successivo fornirgli quel JSON chiedendo di verbalizzarlo in testo continuo – questo può migliorare la coerenza, perché il modello nel primo step si è concentrato sulla struttura logica.

Implementazione con OpenAI (GPT-4/Codex)

Il prompt chaining con GPT-4 è molto comune. Un esempio concreto è la pipeline di **soluzione di problemi complessi** proposta da OpenAI chiamata *"Let's think step by step"* → *"Let's code it"* → *"Let's verify"*. Qui abbiamo almeno tre fasi: prima GPT-4 ragiona e produce un piano o pseudocodice, poi GPT-4 (o Codex) genera codice Python da quel piano, poi eseguiamo il codice e diamo a GPT-4 i risultati perché li interpreti e formuli la risposta finale. Possiamo implementare questo orchestrandolo in un semplice script:

1. call GPT-4 con prompt "Pensa passo passo e formula un piano..." -> output plan;
2. call GPT-4 con prompt "Genera codice Python per eseguire il piano: [plan]" -> output code;
3. eseguire codice, ottenere ad es. `result.txt`;
4. call GPT-4 con prompt "Il codice ha prodotto questo output: [risultati]. Rispondi ora alla domanda iniziale rifacendoti ai risultati." -> output finale.

Questo è un esempio di catena multi-modalità (testo -> codice -> esecuzione -> testo). Per implementare catene lineari pure testuali, si può utilizzare **OpenAI Functions** in modo creativo: definire fasi come funzioni fittizie. Tuttavia, spesso è più semplice gestire tutto via codice orchestratore. GPT-4 essendo costoso conviene usarlo nelle fasi cruciali, mentre per passi meccanici possiamo farli fare a modelli minori o a funzioni programmatiche. Un tipico scenario: generazione di contenuti lunghi. Step1 con GPT-3.5: genera outline; Step2 con GPT-4: per ciascun punto genera sezione; Step3 con GPT-4: unisci sezioni e migliora la fluidità del testo; Step4 con GPT-3.5: correggi grammaticalmente (o persino passare in un correttore dedicato). Un altro scenario: Q&A su documenti. Step1: GPT-3.5 raccoglie fatti chiave dal documento; Step2: GPT-4 formula la risposta usando quei fatti. Così minimizziamo chiamate GPT-4 su tutta la lunghezza. **Codex** spesso lo si usava in catene di codegen: scrivi test -> Codex genera funzione -> esegui test -> Codex corregge funzione. Anche qui è una pipeline iterativa. In generale, le *Chain-of-Thought* a volte vengono implementate come prompt chaining: invece di far generare il CoT e la risposta in un unico prompt (rischio di leak), si può fare: prompt1: "dimmi il ragionamento passo passo" -> output1; prompt2: "dato il ragionamento, qual è la risposta?" -> output2. Così puoi controllare separatamente il ragionamento e la final answer. Questo fu uno stratagemma usato in alcune competizioni per evitare che il modello desse risposte finali sbagliate nonostante il ragionamento corretto.

Esempi

Tra gli esempi più evidenti c'è **GPT-3 Chain-of-thought coaching**: OpenAI nell'uscita di GPT-4 mostrò un esempio di complessa moltiplicazione fatto in catena: prima GPT-4 veniva forzato a scrivere i passaggi (coi calcoli a mano), poi un secondo passo prendeva quei passaggi e li eseguiva ottenendo il risultato corretto. Un altro esempio reale è **AI Dungeon** (un gioco di testo generativo): la comunità sperimentò *Story progression chains* dove un modello generava la prossima scena, un secondo modello generava il dialogo dettagliato basato su quella scena, un terzo rifiniva lo stile, ecc., per avere storie più coerenti. In ambito aziendale, uno scenario tipico è la **generazione di email di marketing**: Step1 con LLM crea una bozza basata su bullet point forniti, Step2 con un altro prompt migliora il tono rendendolo più accattivante, Step3 con una funzione sostituisce placeholder (tipo nome cliente), Step4 un ultimo LLM check di qualità. Ogni step migliora l'output finale. Ci sono anche architetture di chain predefinite come **PAL (Program Assisted Logic)**: il modello prima genera uno pseudocode in linguaggio naturale per un problema, poi quel testo viene interpretato da un parser che esegue calcoli, poi i risultati vanno al modello per generare la risposta. Questo è usato per math word problems e può essere visto come un prompt chain (anche se uno step è eseguito da codice). In sintesi, ogni volta che spezzettiamo un compito complesso in interazioni multiple con l'LLM, stiamo applicando il pattern della Catena di Prompt.

10. Pattern Dibattito Multi-Agente (Ensemble/Debate)

Descrizione

In questo pattern, **più agenti AI vengono messi in dialogo o competizione** fra loro per esplorare diverse soluzioni e giungere a una risposta tramite **consenso, voto o argomentazione** [80] [81]. Ci sono vari sottopattern: nel *Multi-Agent Debate*, due o più agenti sostengono tesi opposte su una domanda e cercano di confutarsi a vicenda, mentre un giudice (che può essere un altro modello o un umano)

decide quale argomentazione è più convincente [80]. Nell'approccio *Ensemble/Voting*, diversi agenti indipendenti producono ciascuno la propria risposta, e poi si applica un meccanismo di voto o consenso per determinare la risposta finale (ad esempio la risposta maggioritaria, o una media delle soluzioni) [81]. L'idea chiave è che la **diversità di opinioni modellistiche** e l'interazione tra agenti può portare a un risultato più robusto: gli agenti possono correggersi a vicenda o portare alla luce aspetti che un singolo agente avrebbe trascurato.

Vantaggi e limiti

Vantaggi: Questo pattern può **migliorare l'accuratezza e l'affidabilità** delle risposte. Se più agenti arrivano indipendentemente alla stessa conclusione, è meno probabile che sia frutto di un'allucinazione (principio del consenso). Nel caso del dibattito, gli agenti portano argomenti pro/contro, il che aiuta a far emergere la verità tramite contraddittorio – ad esempio per domande complesse o ambigue, il dibattito può chiarire i punti chiave e arrivare a una posizione ragionata. Questo meccanismo è stato anche proposto come tecnica di **allineamento** (AI Safety via Debate) in cui due modelli discutono e in teoria le inesattezze logiche vengono smascherate dal contraddittorio. L'**ensemble voting** è efficace nel ridurre errori casuali: se 7 agenti su 10 dicono X e 3 dicono Y, si tende a fidarsi di X (simile al boosting nel machine learning). Anche qualitativamente, un agente può aggiungere dettagli che un altro non conosce, portando a una risposta finale più completa se li si unisce.
Limiti: Coinvolgere più modelli comporta **maggiore costo computazionale** (es: 5 agenti paralleli costano 5 volte un agente). Inoltre, un rischio è il **conformismo**: modelli simili potrebbero generare risposte simili con gli stessi bias, quindi avere 5 agenti clonati non è garanzia di diversità; potremmo ottenere 5 volte la stessa allucinazione. Bisogna quindi che gli agenti abbiano una qualche diversità (diverse istruzioni, inizializzazioni o persino architetture). Nel dibattito, c'è l'incognita di come valutare chi "vince": se il giudice è un modello, potrebbe essere ingannato da retorica convincente ma sbagliata (problema noto: un modello può preferire argomentazioni superficiali ma ben formulate). Se il giudice è umano, si rintroduce un umano-in-loop. Nell'ensemble semplice (voto di maggioranza), c'è il rischio che la maggioranza sbagli perché condividono tutti la stessa base di conoscenza incompleta. Inoltre orchestrare un dibattito è complicato: bisogna assicurarsi che i modelli rispondano l'un l'altro correttamente senza andare fuori tema o degenerare in loop di accordo. C'è il problema del *gradiente vanishing delle responsabilità*: se tutti rispondono, chi integra? Servono strategie per sintetizzare poi le varie risposte (p.es. far discutere finché convergono, o prendere la più votata).

Implementazione con Claude

Con Claude possiamo condurre un dibattito multi-agente sfruttando più istanze oppure facendo roleplay in una singola istanza. **Approccio multi-istanza:** Avviare ad esempio due sessioni separate di Claude (Agent A e Agent B). Porre ad entrambi la stessa domanda o assegnare ruoli (es: A deve sostenere "vero", B "falso" riguardo a un'affermazione). Quindi iterare: prendere la risposta di A, darla in pasto a B chiedendo una confutazione, poi prendere la contro-risposta di B e darla ad A, e così via. Questo scambio può essere ripetuto tot turni (bisogna essere cauti, possono diventare lunghi). Infine, per decidere il vincitore, possiamo coinvolgere un terzo Claude come **giudice**: fornirgli l'intera trascrizione del dibattito e chiedere un verdetto motivato su chi aveva ragione. Claude, avendo politiche di non essere troppo contraddittorio, spesso in realtà tende a trovare compromessi se non istruito a dovere. Quindi conviene incorniciare il compito chiaramente: *"Devi convincere l'altro agente che la tua posizione è corretta"* e dare a ciascuno un ruolo fermo. **Approccio singola istanza (self-debate):** È possibile far sì che un solo Claude simuli più voci. Ad esempio, con opportuno prompt di sistema: *"Oggi faremo un dibattito. Ci sono due agenti: Pro e Contro. Risponderai nel formato: Pro: … ; Contro: … ; (alternando gli interventi per tot turni), infine: Giudice: …"*. Claude potrebbe generare l'intero dibattito in autonomia. Questo è più semplice da implementare, ma meno interessante perché essenzialmente il modello conosce già la verità e scrive un dibattito fittizio a posteriori. Serve più che altro come

esposizione didattica, non come vera scoperta della verità tramite conflitto. Per il pattern ensemble senza dibattito, potremmo semplicemente fare n chiamate parallele a Claude con leggera variazione (ad esempio cambiando il "temperature" per avere risposte diverse, o ponendo la domanda in formulazioni diverse per ottenere prospettive multiple). Poi possiamo prendere tutte le risposte e decidere: o fare un voto maggioranza se convergono, oppure fonderle. La fusione potremmo farla fare a Claude stesso: *"Ecco 3 risposte da 3 agenti: [risposta1] [2] [3]. Metti insieme le idee e fornisci la sintesi finale."*. Claude eccelle di solito in compiti di sintesi comparativa. Questo approccio ensemble migliora il contenuto finale. Infine, Claude con la sua ampia context window potrebbe addirittura gestire 5-6 agenti in una conversazione unica (girandosi da solo), ma c'è il rischio che tenda a collaborare con se stesso più che a contraddire.

Implementazione con OpenAI (GPT-4/Codex)

GPT-4 può essere utilizzato in modalità dibattito con più conversazioni parallele. OpenAI aveva proposto il concetto di **AI debate** come metodo di allineamento: due modelli GPT competono su una domanda di cui uno conosce la risposta corretta (ad es. un puzzle) e l'altro cerca di convincere che una risposta errata sia giusta; un terzo modello valuta. Per implementarlo, si seguono i turni come con Claude multi-istanza. Un dettaglio: GPT-4 a livello di API non può iniziare spontaneamente a contraddire l'altro a meno di istruzioni, quindi occorre definire i ruoli in system prompt per ciascuno: *"Assistant 1, tuo ruolo: sostenere questa tesi… Assistant 2, tuo ruolo opposto…"* e mantenere separate le chat. Poi orchestrare scambi di messaggi copiando le risposte da uno all'altro con prefissi tipo "L'altro agente ha detto: … come controbatti?". E così via. Si può anche fare un *debate in one prompt* come fu mostrato in alcune demo: tipo *"Q: [domanda]. A1: [risposta di model1]. A2: [risposta di model2]. Judge: [valutazione]."* e dare questo tutto a GPT-4 con istruzione *"Riempie le parti mancanti"* avendo definito che A1 e A2 devono fornire argomenti diversi. Ma come per Claude, farlo in un colpo solo fa sì che GPT-4 scriva da solo tesi, antitesi e sintesi, che è utile come output formato, ma non sfrutta la competizione genuina tra modelli con parametri diversi. Una strada efficace è usare modelli **specializzati o con temperature diverse**: ad es. prendere GPT-4 come agente 1, GPT-3.5 come agente 2, per vedere approcci differenti, e GPT-4 come giudice (perché GPT-4 è più bravo a valutare). Oppure prendere lo stesso GPT-4 ma con temperature random alte per generare molteplici risposte diverse (approccio ensemble): se la domanda è difficilissima, potremmo far generare a GPT-4 8 possibili risposte (campionando diversamente) e poi fargli votare quella più plausibile. Codex in questi pattern non era molto usato (tranne forse come "uno degli agenti specializzati in coding" in un ensemble multimodale, ma è un caso limite). Un'altra variante di ensemble, più tecnica: eseguire lo stesso prompt su più modelli (es: GPT-4, Anthropich Claude, Google PaLM) e poi combinare le risposte – questo incarna un *mixture of experts inter-modello*. Non sempre facile da orchestrare, ma a volte fatto manualmente da utenti per confrontare risposte.

Esempi

Un esempio concettuale famoso è la proposta **"AI vs AI Debate"** di OpenAI (Irving et al. 2018) dove due modelli discutevano se una immagine conteneva un determinato oggetto, cercando di ingannare il giudice. Questo per esplorare la veridicità: in molti casi, si è visto che modelli possono rivelare bug di altri modelli. Un esempio pratico attuale: **Google DeepMind** sta sperimentando *"sparring AI"* per MedPaLM (ambito medico) dove un modello propone diagnosi e un secondo modello gioca all'avvocato del diavolo cercando errori o alternative, poi un terzo (umano o AI) decide. In ambito non competitivo ma ensemble: **Bing Chat** a volte combina i risultati di più istanze con diversi prompting (anche se non visibile all'utente). Nell'**ambiente accademico**, c'è il pattern di *"self-consistency"* che è una sorta di ensemble temporale: il modello genera molte chain-of-thought diverse e poi prende la risposta più comune [81] [82]. Questo ha migliorato notevolmente la precisione di ragionamento aritmetico. Ad esempio, per una domanda matematica, fai generare 20 ragionamenti indipendenti a GPT, ciascuno porta a un numero finale; poi vedi quale numero appare più spesso come risultato finale e prendi

quello come risposta. Questo riduce errori casuali dovuti a scelte sbagliate nel ragionamento. Un altro esempio, **HuggingFace HuggingChats** organizzava una volta un dibattito tra due LLM per confrontare chi era più factual. E nella community si fanno spesso esperimenti di *roleplay* come "tre chatGPT in una stanza che discutono di filosofia" – a scopo ricreativo ma illustrano il pattern. Infine, ensemble di specialisti: nell'**assistente JARVIS (HuggingGPT)** c'è un LLM orchestratore che chiede a diversi modelli (visione, NLP, ecc.) output e poi li combina; sebbene non dibattano, è un *Mixture of Experts pattern* che discuteremo sotto, imparentato con il concetto multi-agente dove ogni agente è un esperto e la decisione finale risulta dalla collaborazione di tutti.

11. Pattern Mix di Agenti Specializzati (Mixture of Experts)

Descrizione

Questo pattern prevede di mettere insieme **agenti eterogenei con competenze diverse** (o addirittura modelli di natura diversa: linguistici, visivi, auditoriali, ecc.) per affrontare compiti complessi in modo **collaborativo e multimodale** [83] [9]. Invece di avere un agente generale che fa tutto, si crea una sorta di *team di esperti AI*: ciascun agente è specializzato in un dominio (es. analisi di immagini, generazione di codice, traduzione, calcolo scientifico) e un **agente coordinatore** (o una logica di orchestrazione) suddivide il problema e aggrega le soluzioni. Spesso c'è un *LLM principale* che funge da *controller* (cervello) e invia sottocompiti agli specialisti, poi raccoglie i risultati integrandoli in una risposta coerente [10] [84]. Questo pattern è esemplificato da sistemi come **HuggingGPT** di Microsoft, dove ChatGPT supervisiona molti modelli di HuggingFace (visione, NLP, ecc.) orchestrandoli per risolvere un input multimodale complesso [9] [10].

Vantaggi e limiti

Vantaggi: Permette di sfruttare il **meglio di ogni modello**: un modello di visione per capire immagini, un modello di linguaggio per ragionare, uno di calcolo per risultati numerici, etc. Questo porta a un sistema complessivamente più forte di ciascun singolo modello isolato [9] [10]. La specializzazione significa anche efficienza: ogni sub-agente fa un lavoro mirato invece di appesantire un solo LLM con tutto. Ad esempio, nel multimodale, un singolo mega-modello che faccia sia visione che linguaggio potrebbe essere meno efficiente che due modelli separati ottimizzati. Inoltre, la modularità consente di **aggiornare componenti** facilmente: possiamo rimpiazzare l'esperto di visione con uno più nuovo senza dover riaddestrare l'intero sistema. In termini di risultati, questo pattern favorisce soluzioni **complete e accurate** perché ciascun aspetto del problema è risolto dallo specialista migliore in quel campo, sotto il coordinamento di un LLM che ha visione d'insieme.
Limiti: L'orchestrazione è complessa: il coordinatore deve sapere quale esperto chiamare per ciascun sottocompito e come comporre i risultati. Questo spesso richiede descrizioni formali delle capacità di ciascun agente (ad esempio HuggingGPT richiedeva le "tag" dei modelli HuggingFace per capire quale modello usare per "segmentation" vs "classification") [85] [84]. Se la descrizione è sbagliata o incompleta, il controller può scegliere il modello sbagliato o non sfruttarne appieno le capacità. C'è overhead comunicativo: i dati devono passare tra agenti (ad es. l'immagine -> descrizione testuale -> LLM -> risultato -> etc.), e ogni passaggio può introdurre errori di conversione. Inoltre, dipendere da tanti componenti significa che **il fallimento di uno** può compromettere l'intera catena (serve robustezza e magari fallback: se l'esperto A non risponde, tentarne un altro simile). Dal lato implementativo, c'è anche il costo di eseguire molti modelli diversi, potenzialmente su hardware diverso (GPU per visione, CPU per LLM, ecc.), il che richiede infrastruttura orchestrativa notevole. Infine, l'utente finale potrebbe ricevere output eterogenei: bisogna normalizzare formati e stili. In sintesi, serve un *coordinamento intelligente* (spesso delegato a un LLM) ma questo aggiunge un ulteriore livello di complessità e possibilità di errore.

Implementazione con Claude

Claude può essere impiegato come **agente orchestratore** di altri specialisti, oppure come specialista in un team coordinato da altro. Immaginiamo di usare Claude come orchestratore: possiamo seguire l'approccio di HuggingGPT. Forniamo a Claude una **lista di strumenti/modelli disponibili** con descrizioni. Ad esempio: *"Hai accesso ai seguenti specialisti: 1) VisionAI – input: immagine, output: descrizione; 2) CalcAI – input: formula, output: risultato numerico; 3) TranslateAI – input: testo, output: traduzione."* e così via. Poi se l'utente fa una richiesta come "prendi questa immagine, descrivila, poi traduci la descrizione in francese e calcola quanti caratteri ha la traduzione", Claude deve capire di coinvolgere VisionAI, poi TranslateAI, poi CalcAI (questo è un flusso orchestrato). Realizzarlo concretamente: se abbiamo implementazione per VisionAI (es un endpoint che restituisce descrizione) e TranslateAI ecc., Claude può ragionare e dire: *"Per prima cosa uso VisionAI sull'immagine fornita."* (magari in formato [TOOL] come detto prima). Noi eseguiamo VisionAI, otteniamo "un gatto nero su un tappeto". Claude riceve questo e continua: *"Ora uso TranslateAI sul testo 'un gatto nero su un tappeto' in francese."*. Esegue, otteniamo "un chat noir sur un tapis". Claude: *"Ora uso CalcAI sul testo per contare i caratteri."*. Esegue, otteniamo ad esempio 24 caratteri. Infine Claude risponde all'utente: *"Nell'immagine c'è un gatto nero su un tappeto. In francese: 'un chat noir sur un tapis'. Questa frase ha 24 caratteri."*. Questo esempio illustra come un orchestratore come Claude deve mantenere il *workflow state* e ricordare i risultati man mano [85] [84]. Possiamo aiutare Claude includendo nel prompt uno schema di pianificazione: *"Se il compito richiede più passi, pianifica una sequenza [Passo1 -> Passo2 -> ...] indicando quale specialista usare e con che input."*. Claude magari elenca i passi, e poi possiamo farlo procedere eseguendo e tornando con i risultati. In implementazioni reali, potremmo avere un codice che alimenta progressivamente i risultati a Claude. L'alternativa è usare Claude come uno degli esperti: ad esempio, un sistema orchestrato con GPT-4 come regista potrebbe chiamare Claude come worker per compiti di ragionamento lungo (Claude per la parte di stesura lungo testo perché ha contesto maggiore, GPT-4 per altra parte). In tal caso, dobbiamo predisporre un endpoint per Claude andata/ritorno, e GPT-4 lo userebbe come uno strumento tramite function call. Questo è fattibile ma forse meno comune a livello di design, più probabile orchestrare modelli più specifici (CV, speech) con un LLM centrale. Va detto che l'**API di Claude** al momento non supporta immagini o audio, quindi come specialista in ambito testuale va bene, ma per visione o audio serve chiamare modelli open source o servizi esterni.

Implementazione con OpenAI (GPT-4/Codex)

GPT-4 è spesso usato come orchestratore principale in mixture of experts. **HuggingGPT** implementava proprio ChatGPT (basato su GPT-4) come controller [9] [10]. Il paper mostra che ChatGPT riceve la richiesta utente, decomprime i subtasks, sceglie modelli appropriati su HuggingFace Hub, li invoca (tramite un backend Python) e poi aggrega i risultati. Simulare questo: con OpenAI Functions, potremmo definire una funzione `use_model(model_name, input)` generica; GPT-4 può chiamare `use_model("Vision-Model-123", "<image_bytes>")`, la nostra funzione la esegue su un server con quell'AI e rimanda il risultato. ChatGPT poi decide passo successivo. In mancanza di function call, in HuggingGPT hanno fatto tutto text-based: ChatGPT outputtava una sorta di JSON con plan e modelli, uno script python li leggeva ed eseguiva, poi rimandava risultati che ChatGPT integrava [10] [86]. Per implementazioni custom, spesso si utilizza **LangChain Agents** con tool specializzati: es. un tool per visione (che sotto utilizza un modello come CLIP interrogato con un prompt), un tool per i calcoli (Python), e far girare un agente GPT-4 con quei tool disponibili. GPT-4 allora comportandosi in stile ReAct li userà quando opportuno. Codex era specialista in codice: un mixture of experts scenario è scenario di *Software development agent*: GPT-4 orchestratore divide compiti (generare codice, testare, documentare) e invoca Codex per scrivere effettivamente il codice (che magari eccelle in sintassi), poi GPT-4 integra i risultati (fa il code review, scrive documentazione). Quindi GPT-4 orchestratore, Codex worker. Con function calling, GPT-4 può delegare ad una funzione come `execute_code(code)` e la funzione in backend magari usa un runtime Python, oppure delegare addirittura a un modello code-

davinci a cui passare il snippet da completare. L'integrazione di modelli multi-modalità è un fronte attivo: per esempio OpenAI ha Vision (DALL-E, GPT-4 vision internally) ma se volessi integrare un modello di text-to-speech, definirei una function `tts_engine(text)` e GPT-4 la chiamerebbe quando vuole un output audio. Insomma, GPT-4 funge da *router intelligente* [87] [88]. Un altro aspetto: la *cooperazione tra pari specialisti* senza orchestratore unico – come un team senza manager. Ad esempio, un "agente traduttore" e un "agente correttore" che iterano sul testo migliorandolo (simile al critic-actor ma su specialità diverse). È però più efficiente avere una sorta di manager.

Esempi

HuggingGPT è l'esempio principe: lì hanno risolto compiti come *"prendi questa immagine, descrivila, poi domanda qualcosa su di essa, poi genera un'immagine modificata"* – ChatGPT coordinava modelli di visione e generazione immagini per eseguire ogni step [10] [84]. Altro esempio, **Voyager** (un agente per Minecraft): GPT-4 pianificava le azioni da fare nel gioco, ma delegava l'esecuzione a moduli specifici di navigazione e combattimento (questi moduli interpretavano i comandi di GPT e restituivano stato). Qui GPT-4 era cervello e i moduli fungevano da esperti del dominio "Minecraft". Nel mondo enterprise, si parla di *AI Orchestration* in cui un LLM richiama servizi come "analisi sentiment", "OCR", ecc. Un pattern di mixture of experts molto comune è anche il **retrieval + LLM + calcolatore** con orchestrazione: se l'utente chiede "qual è la media mobile a 7 giorni delle vendite negli ultimi 3 mesi?", l'agente potrebbe: interrogare database (esperto DB), ottenere numeri, poi eseguire calcolo (esperto analitico), poi formulare risposta (LLM linguistico). Questo può essere concepito come un orchestratore che chiama due tool diversi in sequenza. Un ultimo esempio è **AutoGen di Microsoft**: è un framework dove definisci agenti come classi Python (ad esempio "PythonExpert", "ExcelAgent", "PlannerAgent") e li lanci in un gruppo. Questi agenti comunicano via un autogenerato canale (un prompt condiviso) e l'AutoGen li fa lavorare insieme. Si è visto un caso in cui un agente Python generava code, un agente ChatGPT chiedeva spiegazioni su requisiti, etc., in tandem. Questo mostra come un mix di agenti con ruoli specializzati può co-creare una soluzione.

Fonti: Questi pattern sono discussi in vari articoli e implementazioni, ad esempio Shoubhik Ghosh (2025) li sintetizza nei design pattern Reflection, ReAct, Planner-Executor, Manager-Worker, Critic, Tool-Use, Memory, Human-in-Loop, Guardrail [89]; Manas Mallik (2025) elenca ReAct, Agentic RAG, Reflexion, Hierarchical, Orchestrator-Worker, Prompt Chaining, Parallelization, Evaluator-Optimizer, Multi-Agent Debate, Mixture of Agents [90] [91]. Implementazioni open-source come LangChain, AutoGPT, HuggingGPT e others mostrano come applicare questi schemi nella pratica [92] [9]. Sperimentare con tali pattern permette di costruire agenti AI più robusti, **collaborativi** e capaci di affrontare problemi complessi in modo strutturato e comprensibile.

---

[1] [3] [4] [5] [6] [7] [8] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20] [21] [28] [29] [30] [31] [32] [33] [41] [42] [47] [48] [63] [64] [65] [66] [67] [70] [71] [74] [75] [76] [89] Design Patterns in Agentic AI: The Blueprint for Building Autonomous Intelligence | by Shoubhik Ghosh | Nov, 2025 | Medium
https://medium.com/@shoubhikghosh83/design-patterns-in-agentic-ai-the-blueprint-for-building-autonomous-intelligence-033853e1e0e5

[2] [79] [80] [83] [90] [91] Unlocking AI Autonomy: The 11 Must-Know Design Patterns in Agentic AI
https://www.linkedin.com/pulse/unlocking-ai-autonomy-11-must-know-design-patterns-agentic-mallik-2uase

[9] [10] [84] [85] [86] HuggingGPT: Revolutionizing Complex AI Task Management with LLM Integration | Neural Notes
https://neuralnotes.tech/hugginggpt-ba3af0decc4a?gi=005d9f81a750

22 23 24 25 26 27 37 38 The Reflection Pattern: How Self-Critique Makes AI Smarter | by Vishwajeetv | Medium
https://medium.com/@vishwajeetv2003/the-reflection-pattern-how-self-critique-makes-ai-smarter-035df3b36aae

34 35 36 39 40 68 69 Reflection Agents
https://blog.langchain.com/reflection-agents/

43 44 45 46 49 50 51 52 53 ReAct Prompting | Prompt Engineering Guide
https://www.promptingguide.ai/techniques/react

54 55 56 57 58 59 60 61 62 77 78 92 Auto-GPT Development: The Next Leap in Autonomous AI Agents | by Agents Architects | Medium
https://medium.com/@agentsarchitects/auto-gpt-development-the-next-leap-in-autonomous-ai-agents-d03c34f1da05

72 Retrieval-augmented generation - Wikipedia
https://en.wikipedia.org/wiki/Retrieval-augmented_generation

73 What is retrieval-augmented generation (RAG)? - IBM Research
https://research.ibm.com/blog/retrieval-augmented-generation-RAG

81 82 87 88 Exploring Agentic Workflow Patterns - DEV Community
https://dev.to/dpaluy/exploring-agentic-workflow-patterns-312a